



LINE - 2 - JIRA

การพัฒนาระบบจัดเก็บข้อมูลแจ้งปัญหาอัตโนมัติสำหรับทีม App Support
สำหรับบริษัท เคแอลเอ็น ซีพอร์ต จำกัด

โดย

นายภูวดล พูลพัฒน์ รหัสนิสิต 6530202790

โครงการสหกิจศึกษานี้เป็นส่วนหนึ่งของการศึกษาตามหลักสูตร
วิทยาศาสตร์บัณฑิต (สาขาเทคโนโลยีสารสนเทศ) คณะวิทยาศาสตร์ ศรีราชา
มหาวิทยาลัยเกษตรศาสตร์ วิทยาเขตศรีราชา
ปีการศึกษา 2568

ใบรับรองสหกิจศึกษา

คณะวิทยาศาสตร์ ศรีราชา

ภาควิชา วิทยาการคอมพิวเตอร์และสารสนเทศ

ปริญญาวิทยาศาสตรบัณฑิต สาขาเทคโนโลยีสารสนเทศ

LINE-2-JIRA: การพัฒนาระบบจัดเก็บข้อมูลแจ้งปัญหาอัตโนมัติสำหรับทีม App Support

สำหรับบริษัทเคแอลเอ็น ซีพอร์ต จำกัด

นายภูวดล พูลพิพัฒน์ รหัสนิต 6530202790

ได้รับการตรวจสอบและอนุมัติ ให้เป็นส่วนหนึ่งของการศึกษาตามหลักสูตร

กรรมการและอาจารย์ที่ปรึกษา : _____ (ผศ.ดร. จีรวรรณ เจริญสุข)

กรรมการ : _____ (ผศ.ดร. สุภาพร บรรดาศักดิ์)

ประธานหลักสูตร : _____ (ผศ.ดร. ฉัตรชัย เกษมทวีโชค)

วันที่ 3 เมษายน 2569

กิตติกรรมประกาศ

กิตติกรรมประกาศฉบับนี้ ผู้จัดทำโครงการขอกราบขอบพระคุณอาจารย์ที่ปรึกษาโครงการ ที่ได้กรุณาให้คำแนะนำ ชี้แนะแนวทาง ตลอดจนตรวจสอบและแก้ไขข้อบกพร่องต่าง ๆ ของโครงการด้วยความเอาใจใส่ ซึ่งเป็นประโยชน์อย่างยิ่งต่อการจัดทำโครงการฉบับนี้ให้สำเร็จลุล่วงไปได้ด้วยดี

ข้าพเจ้าได้มาปฏิบัติงานในโครงการสหกิจศึกษา ณ บริษัท KLN Seaport ส่งผลให้ข้าพเจ้าได้รับความรู้และประสบการณ์ต่าง ๆ ที่มีค่ามากมาย โดยเฉพาะอย่างยิ่งในการพัฒนาระบบ “LINE-2-JIRA: ระบบจัดเก็บข้อมูลแจ้งปัญหาอัตโนมัติสำหรับทีม App Support” ข้าพเจ้าใคร่ขอขอบพระคุณบุคคลและกลุ่มคณะที่มีส่วนเกี่ยวข้อง ดังนี้:

- **บริษัท KLN Seaport:** ที่ให้โอกาสข้าพเจ้าได้เข้ามาเรียนรู้การทำงานในสภาพแวดล้อมจริง และสนับสนุนทรัพยากรต่าง ๆ ในการดำเนินโครงการ
- **ทีม App Support:** ในฐานะผู้ใช้งานหลัก ที่ได้ให้ข้อมูลเกี่ยวกับปัญหาการจัดการข้อมูลแชทที่กระจุกกระจาย และช่วยให้ข้าพเจ้าวิเคราะห์รูปแบบข้อมูล (Data Pattern) เพื่อนำมาออกแบบระบบให้สอดคล้องกับการทำงานหน้างานจริง
- **พี่ ๆ ทีมที่ปรึกษาและพนักงานทุกท่าน:** ที่ได้ให้คำแนะนำช่วยเหลือ และสละเวลาให้คำปรึกษาในการจัดทำรายงานฉบับนี้ รวมถึงการปรับปรุง Logic ของระบบ n8n ให้มีความแม่นยำและปลอดภัยตามมาตรฐานขององค์กร

ข้าพเจ้าใคร่ขอขอบพระคุณผู้ที่มีส่วนเกี่ยวข้องทุกท่านที่มีส่วนร่วมในการให้ข้อมูลและเป็นที่ปรึกษาในการทำรายงานฉบับนี้จนเสร็จสมบูรณ์ ตลอดจนให้การดูแลและให้ความรู้อันเป็นพื้นฐานสำคัญในการปฏิบัติงานจริง ข้าพเจ้าขอขอบพระคุณเป็นอย่างสูงไว้ ณ โอกาสนี้

ภูวดล พูลพิพัฒน์

3 เมษายน 2569

ชื่อเรื่อง (ไทย)	LINE-2-JIRA: ระบบจัดเก็บข้อมูลแจ้งปัญหาอัตโนมัติสำหรับทีม App Support
(English)	Automated Incident Logging System for App Support Team
ชื่อผู้เขียน	นายภูวดล พูลพิพัฒน์ รหัสนิต 6530202790
ชื่อปริญญา	วิทยาศาสตรบัณฑิต (เทคโนโลยีสารสนเทศ)
ภาควิชา	วิทยาการคอมพิวเตอร์และสารสนเทศ
อาจารย์ที่ปรึกษา	ผศ.ดร. จีรพรรณ เจริญสุข
สถานศึกษา	มหาวิทยาลัยเกษตรศาสตร์ วิทยาเขต ศรีราชา

บทคัดย่อ

โครงการนี้พัฒนาโครงการ “LINE-2-JIRA: การพัฒนาระบบจัดเก็บข้อมูลแจ้งปัญหาอัตโนมัติสำหรับทีม App Support” ของบริษัท KLN Seaport เพื่อลดภาระงานบันทึกข้อมูลและเพิ่มประสิทธิภาพการติดตามปัญหาจาก Line Official Account โดยใช้เครื่องมือ n8n Workflow Automation เชื่อมต่อกับ Line Messaging API เพื่อวิเคราะห์รูปแบบข้อมูล (Data Pattern) และดำเนินการสร้าง Ticket บนระบบ Jira Service Management ในสภาพแวดล้อม UAT (User Acceptance Test) โดยอัตโนมัติ

ขอบเขตการทำงานครอบคลุมข้อมูลการแจ้งปัญหาทั้งฝั่ง Container (CT) และ Conventional (CV) โดยระบบใช้เทคโนโลยี Google Gemini 2.5 Flash ในการวิเคราะห์และจำแนกประเภทเคสออกเป็น 12 กลุ่มตามลักษณะงานขององค์กร พร้อมทั้งสกัดข้อมูลสำคัญให้อยู่ในรูปแบบข้อมูลโครงสร้าง (Structured Data) เพื่อบันทึกลงใน Google Sheets และสร้าง Issue ในระบบ Jira พร้อมกำหนดค่าความเร่งด่วนและหมวดหมู่ที่เกี่ยวข้องทันที

ผลการดำเนินงานช่วยลดความล่าช้าในการสรุปปัญหาในแต่ละวัน ลดความผิดพลาดจากการตกหล่นของข้อมูลผ่านระบบป้องกันการแจ้งปัญหาซ้ำ (Duplicate Prevention) และเป็นต้นแบบการใช้ Workflow Automation ร่วมกับ AI เพื่อยกระดับงานบริการด้านเทคโนโลยีสารสนเทศขององค์กรให้มีประสิทธิภาพและเป็นระบบมากยิ่งขึ้น

สารบัญ

เรื่อง	ลำดับ
ใบรับรองสหกิจศึกษา.....	ก
กิตติกรรมประกาศ	ข
บทคัดย่อ.....	ค
สารบัญ	ง
สารบัญรูปภาพ	จ
บทที่ 1	1
บทนำ	1
1.1 ที่มาและความสำคัญของโครงการ	1
1.2 วัตถุประสงค์ของโครงการ	1
1.3 ขอบเขตของโครงการ	1
1.4 เทคโนโลยีที่ใช้ในการพัฒนาโครงการ.....	2
1.5 ประโยชน์ที่คาดว่าจะได้รับ	3
1.6 แผนการดำเนินงาน	3
1.7 นิยามศัพท์เฉพาะ	4
บทที่ 2.....	5
ทฤษฎีและเทคโนโลยีที่เกี่ยวข้อง	5
2.1 LINE Messaging API	5
2.2 n8n Workflow Automation	6
2.3 Regular Expression (Regex)	7
2.4 Google Gemini AI และ Groq (AI Language Model).....	7
2.5 Jira Service Management	8
2.6 Google Sheets	8
บทที่ 3.....	10
การออกแบบและพัฒนาระบบ.....	10
3.1 สถาปัตยกรรมระบบ (System Architecture)	10

3.2	การออกแบบ Workflow	11
3.3	การออกแบบประเภทเคส (Case Type Design)	14
บทที่ 4.....		16
	ผลการดำเนินงาน	16
4.1	ผลลัพธ์ที่ได้จากระบบ.....	16
4.2	ปัญหาและการแก้ไขระหว่างการพัฒนา	26
4.3	การทดสอบระบบ.....	26
บทที่ 5.....		27
	สรุปและข้อเสนอแนะ	27
5.1	สรุปผลการดำเนินงาน.....	27
5.2	ประสบการณ์และความรู้ที่ได้รับ.....	27
5.3	ข้อจำกัดของระบบ	27
5.4	ข้อเสนอแนะสำหรับการพัฒนาในอนาคต.....	28
5.5	บทสรุป	28
บรรณานุกรม		29

สารบัญรูปภาพ

เรื่อง	ลำดับ
ภาพที่ 2. 1 ตัวอย่างหน้าจอ การแสดงผลตอบกลับข้อความจาก BOT ผ่านช่องทางไลน์	5
ภาพที่ 2. 2 ตัวอย่างการวาง NODE และการทำงานของระบบ	6
ภาพที่ 2. 3 CODE ตัวอย่างการ REGEX หา PREFIX #แจ้งเคส	7
ภาพที่ 2. 4 ตัวอย่าง AI AGENT ที่ใช้ในการประมวลการแจ้งปัญหาของ USER	7
ภาพที่ 2. 5 ตัวอย่าง SYSTEM DASH BOARD JIRA SERVICE MANAGEMENT	8
ภาพที่ 2. 6 ตัวอย่าง GOOGLE SHEET ที่ใช้ในการเก็บ LOG ของ ปัญหาที่ USER แจ้งเข้ามาเพื่อแก้ปัญหาการแจ้งเคส ซ้ำ (หากมีข้อความเดิมภายใน 10 ที่ และมาจาก USER คนเดิมจะไม่สารถแจ้งซ้ำได้).....	8
ภาพที่ 2. 7 ตัวอย่าง GOOGLE SHEET ที่เป็น STORAGE เพื่อเก็บการสร้างตัว TICKET แล้ว	9
ภาพที่ 2. 8 ตัวอย่างโครงสร้างระบบ	10

ภาพที่ 2. 9 WORKFLOW DIAGRAM	13
ภาพที่ 2. 10 ตัวอย่าง DATA FLOW LEVEL 0.....	14
ภาพที่ 2. 11 ตัวอย่าง DATA FLOW DIAGRAM LEVEL 1	15
ภาพที่ 2. 12 ตัวอย่าง USE CASE FLOW	15
ภาพที่ 2. 13 ตัวอย่างการรับข้อความจาก LINE GROUP และการ REPLY	16
ภาพที่ 2. 14 ตัวอย่างการพิมพ์ข้อความอื่นที่ไม่ใช่ #แจ้งเคส	17
ภาพที่ 2. 15 ตัวอย่างการทำงานของระบบของข้อความที่ไม่ใช่ #แจ้งเคส.....	17
ภาพที่ 2. 16 ตัวอย่าง การทำงานตรวจข้อความที่ไม่มี #แจ้งเคส	18
ภาพที่ 2. 17 ตัวอย่างการตอบกลับของ BOT กรณีรับปัญหาแต่ยังขาดข้อมูล.....	18
ภาพที่ 2. 18 ตัวอย่างการทำงานของรับการตอบกลับ USER	19
ภาพที่ 2. 19 ตัวอย่าง NODE AI AGENT	19
ภาพที่ 2. 20 ตัวอย่าง PROMPT AI.....	20
ภาพที่ 2. 21 ตัวอย่าง MODEL GOOGLE GEMINI.....	21
ภาพที่ 2. 22 ตัวอย่าง MODEL GROQ	21
ภาพที่ 2. 23 ตัวอย่าง STRUCTURED OUTPUT.....	21
ภาพที่ 2. 24 ตัวอย่าง NODE การสร้าง TICKET	22
ภาพที่ 2. 25 ตัวอย่าง การสร้าง TICKET	22
ภาพที่ 2. 26 ตัวอย่างการ เริ่ม START PROCESS.....	23
ภาพที่ 2. 27 ตัวอย่าง การเปลี่ยน STATUS ของ TICKET ให้อยู่ในการรอผู้รับผิดชอบปัญหานี้เข้ามา START PROCESS ด้วยตัวเอง.....	23
ภาพที่ 2. 28 ตัวอย่างการเปิด TICKET ใน JIRA สำเร็จ	24
ภาพที่ 2. 29 ตัวอย่างข้อมูลที่รับมาจาก N8N ก็จะบันทึกตาม หัวข้อที่รับข้อความมาจาก USER.....	24
ภาพที่ 2. 30 ตัวอย่าง การทำงานการป้องกันการเคสซ้ำ.....	25
ภาพที่ 2. 31 ตัวอย่าง การบันทึกข้อมูลการสร้าง TICKET สำเร็จ ไว้ใน GOOGLE SHEET	25

บทที่ 1

บทนำ

1.1 ที่มาและความสำคัญของโครงการ

ปัจจุบันทีม Application Support ของบริษัท KLN Seaport รับแจ้งปัญหาการใช้งานระบบต่าง ๆ ผ่านทางแอปพลิเคชัน LINE เป็นหลัก ซึ่งพบปัญหาสำคัญคือข้อมูลการแจ้งเหตุมีลักษณะกระจัดกระจายและไม่มีรูปแบบที่ตายตัว (Unstructured Data) ทำให้เจ้าหน้าที่ต้องใช้เวลาในการคัดกรองและบันทึกข้อมูลด้วยตนเอง ส่งผลให้เกิดความล่าช้าและเสี่ยงต่อการตกหล่นของข้อมูล

จากปัญหาดังกล่าว ผู้จัดทำจึงพัฒนาโครงการ "LINE-2-JIRA: ระบบจัดเก็บข้อมูลแจ้งปัญหาอัตโนมัติสำหรับทีม App Support" โดยใช้เครื่องมือ n8n เป็นตัวกลางจัดการ Workflow เพื่อรับข้อมูลจาก LINE Official Account วิเคราะห์ประเภทและรายละเอียดปัญหาด้วย AI (Google Gemini และ Groq) แล้วสร้าง Issue บน Jira Service Management และบันทึกข้อมูลลง Google Sheets โดยอัตโนมัติ มุ่งเน้นการพัฒนาและทดสอบระบบในสภาพแวดล้อม UAT เพื่อลดภาระงานซ้ำซ้อนของทีม

1.2 วัตถุประสงค์ของโครงการ

- 1.2.1 เพื่อพัฒนาระบบรับแจ้งปัญหาผ่าน LINE Official Account ที่สามารถคัดกรองและวิเคราะห์ข้อมูลได้อัตโนมัติด้วย AI
- 1.2.2 เพื่อจัดรูปแบบข้อมูลแชทที่กระจัดกระจายให้เป็นข้อมูลที่มีโครงสร้าง (Structured Data) พร้อมระบุประเภทเคสและ Field ที่สำคัญ
- 1.2.3 เพื่อสร้าง Jira Issue และบันทึกข้อมูลลง Google Sheets โดยอัตโนมัติแทนการบันทึกด้วยมือ
- 1.2.4 เพื่อเพิ่มความรวดเร็วและลดความผิดพลาดในการรวบรวมข้อมูลของทีม Application Support

1.3 ขอบเขตของโครงการ

- 1.3.1 **ขอบเขตด้านการทำงาน:** พัฒนาการเชื่อมต่อ LINE Messaging API กับ n8n เพื่อรับข้อความแจ้งปัญหา วิเคราะห์ด้วย AI จำแนก 12 ประเภทเคส ได้แก่ SUGAR, WS, WH, COIL, WOODCHIP, TAPIOCA, FREEZONE, GC, PASSWORD, KIOSK, USER_IT และ BARCODE แล้วสร้าง Jira Issue และบันทึกข้อมูลลง Google Sheets โดยอัตโนมัติ
- 1.3.2 **สภาพแวดล้อมที่ใช้:** ทดสอบระบบในสภาพแวดล้อม UAT (User Acceptance Test) เพื่อความปลอดภัยของข้อมูล
- 1.3.3 **ข้อจำกัด:** ระบบมีข้อจำกัดที่ควรรับทราบ 5 ประการ ได้แก่
 - 1.3.3.1 ยังไม่เชื่อมต่อกับ Jira Production โดยตรง ทดสอบเฉพาะใน UAT Environment เท่านั้น

- 1.3.3.2 รองรับเฉพาะข้อความตัวอักษร (Text Message) ไม่รองรับรูปภาพ ไฟล์แนบ หรือ Sticker
- 1.3.3.3 ความแม่นยำของ AI ขึ้นอยู่กับความชัดเจนของข้อความ หากข้อความกำกวม AI อาจจำแนกประเภทผิดพลาดได้
- 1.3.3.4 การใช้ Google Sheets เป็น Cache Storage มีข้อจำกัดด้าน Latency เมื่อข้อมูลสะสมมากหรือมีข้อความเข้าพร้อมกันหลายรายการ
- 1.3.3.5 Case Type ทั้ง 12 ประเภทกำหนดไว้ใน System Prompt ของ AI หากองค์กรเพิ่มระบบงานใหม่ ต้องปรับปรุง Prompt และ Mapping Table ด้วยตนเอง

1.4 เทคโนโลยีที่ใช้ในการพัฒนาโครงการ

- 1.4.1 **LINE Official Account & Messaging API** ใช้เป็นส่วนหน้าของระบบ (Frontend) เพื่อรับข้อความแจ้งปัญหาจากผู้ใช้งานในกลุ่ม LINE โดยอาศัยเทคโนโลยี Messaging API ในการเชื่อมต่อแบบ Webhook เพื่อส่งผ่านข้อมูลให้ผู้ใช้งานพิมพ์ไปยัง n8n แบบ Real-time
- 1.4.2 **n8n Workflow Automation** เป็นเครื่องมือหลักในการจัดการกระบวนการทำงาน (Middleware) รูปแบบ Low-code ทำหน้าที่เป็นตัวกลางรับ Webhook จาก LINE แล้วประมวลผลผ่านโหนด (Node) ต่าง ๆ รวม 33 Node ครอบคลุมตั้งแต่การกรองข้อมูล การตรวจสอบ Duplicate การวิเคราะห์ด้วย AI จนถึงการสร้าง Jira Issue และแจ้งเตือนทีม IT
- 1.4.3 **Regular Expression (Regex)** ใช้ใน Prepare Data Node เพื่อตรวจสอบ Prefix #แจ้งเคส และ Normalize ข้อมูลก่อนส่งให้ AI ประมวลผล รองรับการจัดการอักขระพิเศษ Unicode Non-Breaking Space และช่องว่างซ้ำที่อาจเกิดจากการพิมพ์บนมือถือ
- 1.4.4 **Google Gemini AI และ Groq (AI Language Model)** โครงการนี้ใช้ Large Language Model (LLM) 2 ตัวเป็น AI Engine ภายใน AI Agent Node ได้แก่ Google Gemini จาก Google DeepMind เป็นโมเดลหลักสำหรับวิเคราะห์ประเภทเคส (case_type) ตรวจสอบความครบถ้วนของข้อมูล (is_complete) และสกัด Field ต่าง ๆ จากข้อความแชท โดยมี Groq (Llama 3.3 70B) เป็น Fallback Model เมื่อ Gemini ไม่สามารถตอบสนองได้ทันเวลา ผลลัพธ์ส่งออกเป็น JSON Structure มาตรฐาน
- 1.4.5 **Jira Service Management** ระบบบริหารจัดการ Incident ที่พัฒนาโดย Atlassian ใช้เป็น Output หลักของระบบ เมื่อข้อมูลเคสครบถ้วน n8n จะสร้าง Jira Issue โดยอัตโนมัติพร้อมกำหนด Priority (1 Critical / 2 Important / 3 Standard), Category (KWMS / KPWMS / General Support) และ Summary ตามประเภทเคส จากนั้น Transition สถานะ On Hold → In Progress ทำให้ทีม IT ติดตามงานได้ทันที

1.4.6 Google Sheets ใช้ในโครงการนี้ 2 บทบาท ได้แก่ (1) Duplicate Cache Store ตรวจสอบเคสเข้ามาใน 10 นาที ผ่าน Node Read/Write Duplicate Log และ (2) Data Log บันทึกข้อมูลทุกเคสผ่าน Node Save to Excel ประกอบด้วย Impact, Priority, Category, Summary, Reporter, Group, Received Time และ Jira Ticket Key

1.5 ประโยชน์ที่คาดว่าจะได้รับ

- 1.5.1 ลดภาระงาน Manual** ของเจ้าหน้าที่ Application Support ที่เดิมต้องคัดลอกรายละเอียดจาก LINE ไปบันทึกในระบบอื่นด้วยตนเอง ระบบจะทำหน้าที่วิเคราะห์และบันทึกข้อมูลให้โดยอัตโนมัติ ทำให้เจ้าหน้าที่สามารถใช้เวลาไปกับการแก้ไขปัญหาเชิงเทคนิคได้อย่างเต็มที่
- 1.5.2 ข้อมูลการแจ้งปัญหาถูกจัดเก็บอย่างเป็นระเบียบ** และสามารถตรวจสอบย้อนหลังได้ทันทีผ่าน Google Sheets และ Jira ช่วยให้ข้อมูลมีมาตรฐานเดียวกัน ลดปัญหาการตกหล่นของข้อมูลที่เกิดขึ้นในกลุ่มแชท
- 1.5.3 เป็นต้นแบบการนำ Workflow Automation และ AI มาประยุกต์ใช้** กระบวนการบริการไอทีในองค์กร โครงการนี้เป็นโครงการนำร่องที่พิสูจน์ประสิทธิภาพของการใช้ n8n ร่วมกับ AI LLM ซึ่งสามารถต่อยอดสู่การเพิ่ม Case Type ใหม่ รองรับการส่งรูปภาพ หรือเชื่อมต่อกับ Jira Production ในอนาคต

1.6 แผนการดำเนินงาน

แผนการดำเนินการ	2568		2569			
	พ.ย.	ธ.ค.	ม.ค.	ก.พ.	มี.ค.	เม.ย.
1. ศึกษาโครงสร้างองค์กร และวิเคราะห์ปัญหาการจัดการข้อมูลแชท	←→		←→			
2. ออกแบบโครงสร้างระบบ (Architecture) และกำหนดรูปแบบข้อมูล (Data Pattern)		←→		←→		
3. พัฒนาการเชื่อมต่อ LINE Messaging API เข้ากับระบบ n8n			←→		←→	
4. ทดสอบการบันทึกข้อมูลอัตโนมัติลง Excel ในสภาพแวดล้อม UAT (ปัจจุบัน)				←→	←→	
5. พัฒนาส่วนต่อขยายเพื่อเชื่อมต่อกับระบบ JIRA					←→ ←→	
6. สรุปผลการดำเนินงาน ทดสอบระบบ และจัดทำรายงานฉบับสมบูรณ์					←→ ←→	

ตารางที่ 1 แผนดำเนินการ

■ เส้นที่ 1 (สีน้ำเงิน) = ระยะเวลาที่วางแผนไว้ ■ เส้นที่ 2 (เขียว) = ระยะเวลาที่ปฏิบัติงานจริง

หมายเหตุ: เส้นที่แรงสีน้ำเงิน = แผนที่วางไว้ เส้นที่แรงสีเขียว = ระยะเวลาจริงที่ดำเนินการ หากไม่มีสีส้มหมายความว่าดำเนินการตรงตามแผน

1.7 นิยามศัพท์เฉพาะ

- 1.7.1 **LINE Official Account (LINE OA):** บัญชี LINE สำหรับการสื่อสารในเชิงธุรกิจที่รองรับการเชื่อมต่อกับระบบภายนอกผ่าน API
- 1.7.2 **n8n Workflow Automation:** เครื่องมือสำหรับจัดการ Workflow และเชื่อมต่อบริการต่าง ๆ โดยเน้นการประมวลผลข้อมูลอัตโนมัติแบบ Low-code
- 1.7.3 **Structured Data:** ข้อมูลที่มีรูปแบบการจัดเก็บที่ชัดเจนและเป็นระเบียบ เพื่อให้ง่ายต่อการนำไปประมวลผลต่อ
- 1.7.4 **Unstructured Data:** ข้อมูลที่ไม่มีรูปแบบโครงสร้างที่แน่นอน เช่น ข้อความแชททั่วไปที่ผู้ใช้งานพิมพ์สื่อสารกัน
- 1.7.5 **Webhook:** กลไกการส่งข้อมูลจากระบบหนึ่งไปยังอีกระบบหนึ่งแบบ Real-time เมื่อเกิดเหตุการณ์ที่กำหนด เช่น เมื่อมีข้อความเข้า LINE
- 1.7.6 **UAT (User Acceptance Test):** สภาพแวดล้อมสำหรับทดสอบระบบเพื่อตรวจสอบความถูกต้องก่อนนำไปใช้งานในระดับ Production
- 1.7.7 **AI Agent:** โหนดใน n8n ที่เชื่อมต่อกับ LLM เพื่อวิเคราะห์ข้อความและส่งออกผลลัพธ์ในรูปแบบ JSON
- 1.7.8 **LLM (Large Language Model):** โมเดลภาษาขนาดใหญ่ที่ใช้วิเคราะห์และสร้างข้อความ ในโครงการนี้ใช้ Google Gemini และ Groq
- 1.7.9 **Case Type:** ประเภทของเคสแจ้งปัญหา ระบบรองรับ 12 ประเภท เช่น SUGAR, WS, WH, PASSWORD, KIOSK เป็นต้น
- 1.7.10 **CT (Container):** ส่วนงานที่เกี่ยวข้องกับการให้บริการด้านตู้คอนเทนเนอร์และสินค้าที่บรรจุในตู้
- 1.7.11 **CV (Conventional):** ส่วนงานที่เกี่ยวข้องกับการให้บริการขนส่งสินค้าแบบเทกองหรือสินค้าเฉพาะประเภท

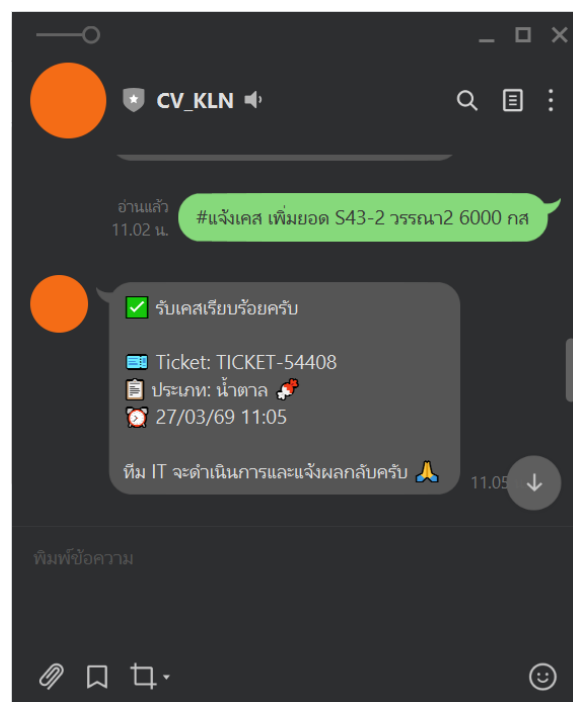
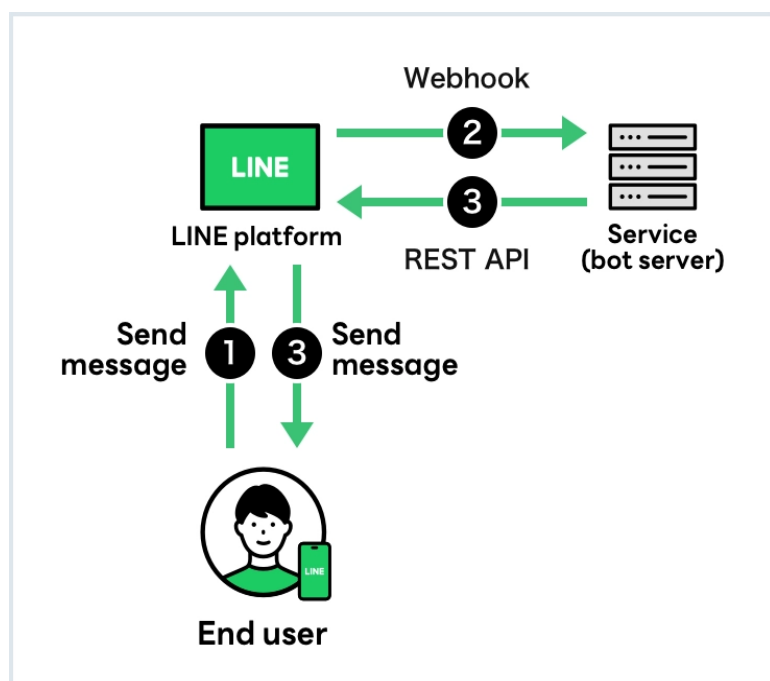
บทที่ 2

ทฤษฎีและเทคโนโลยีที่เกี่ยวข้อง

2.1 LINE Messaging API

LINE Messaging API เป็น API ที่พัฒนาโดย LINE Corporation สำหรับให้ธุรกิจสร้าง LINE Official Account (LINE OA) ที่สามารถสื่อสารกับผู้ใช้งานได้อย่างอัตโนมัติ โดยระบบทำงานบนหลักการของ Webhook ซึ่งเมื่อผู้ใช้งานส่งข้อความเข้ามาถึง LINE OA ระบบจะส่งข้อมูลเหตุการณ์ (Event) นั้นไปยัง URL ที่กำหนดแบบ Real-time (LINE Corporation, 2568)

ในโครงการนี้ LINE Messaging API ทำหน้าที่เป็น Frontend ของระบบรับข้อความแจ้งปัญหาจากเจ้าหน้าที่และผู้ใช้งานภายในกลุ่ม LINE แล้วส่งต่อข้อมูลไปยัง n8n ผ่านกลไก Webhook เพื่อประมวลผลต่อไป (LINE Corporation, 2568)



ภาพที่ 2. 1 ตัวอย่างหน้าจอ การแสดงผลตอบกลับข้อความจาก BOT ผ่านช่องทางไลน์

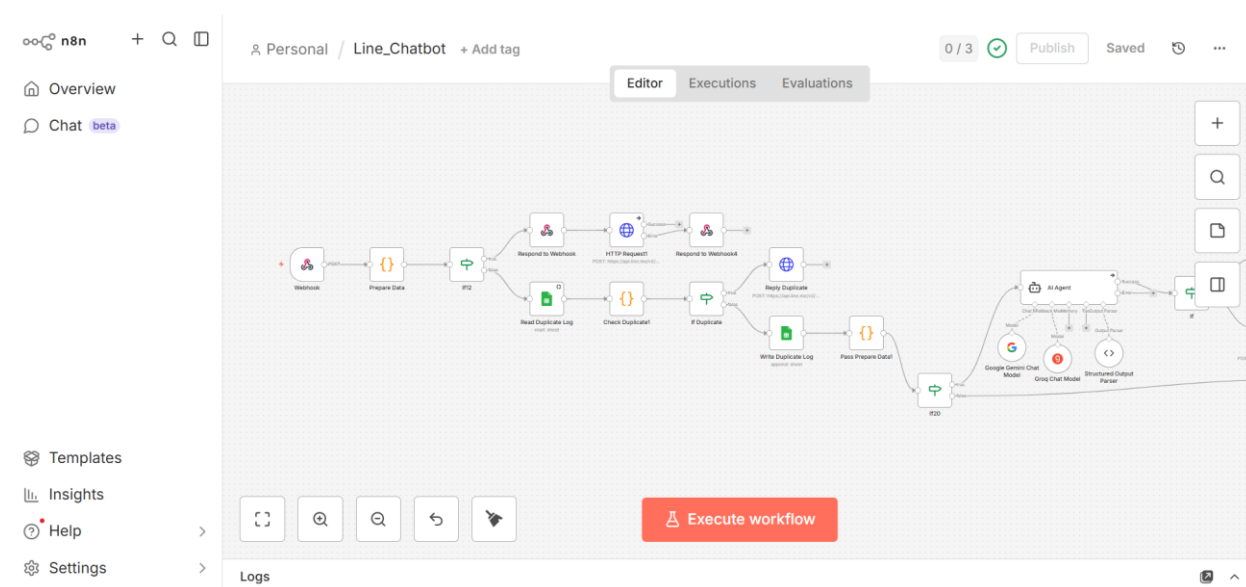
2.2 n8n Workflow Automation

n8n เป็นเครื่องมือ Workflow Automation แบบ Open-source ที่รองรับการทำงานในรูปแบบ Low-code โดยผู้ใช้งานสามารถสร้าง Workflow ด้วยการลากเชื่อมต่อโหนด (Node) ต่าง ๆ เข้าด้วยกัน โดยไม่จำเป็นต้องเขียนโปรแกรมทั้งหมดตั้งแต่ต้น (n8n GmbH, 2568)

n8n รองรับการทำงานเชื่อมต่อกับบริการภายนอกมากกว่า 400 รายการ อาทิ Google Sheets, Jira, LINE API, Gemini AI และอื่น ๆ ในโครงการนี้ n8n ทำหน้าที่เป็น Middleware หรือตัวกลางประมวลผลทั้งหมด ตั้งแต่การรับข้อมูลจาก LINE ไปจนถึงการบันทึกข้อมูลลง Google Sheets และสร้าง Issue บน Jira (n8n GmbH, 2568)

โหนดที่ใช้งานหลักในโครงการมีดังนี้:

- Webhook Node: รับข้อมูล HTTP POST จาก LINE Messaging API เมื่อมีข้อความเข้า
- Code Node: เขียน JavaScript เพื่อประมวลผลและ Transform ข้อมูล
- If Node: แยกทิศทางการประมวลผลตามเงื่อนไขที่กำหนด
- HTTP Request Node: ส่ง API Call ไปยัง LINE และบริการภายนอก
- Google Sheets Node: อ่านและบันทึกข้อมูลลง Google Sheets
- Jira Node: สร้างและจัดการ Issue บน Jira Service Management
- AI Agent Node: เรียกใช้งาน AI Model เพื่อวิเคราะห์ข้อความแบบ Intelligent



ภาพที่ 2. 2 ตัวอย่างการวาง Node และการทำงานของระบบ

2.3 Regular Expression (Regex)

Regular Expression หรือ Regex คือรูปแบบ Pattern ที่ใช้ค้นหาและสกัดข้อความจาก String โดยอาศัยการกำหนดรูปแบบอักขระ (Character Pattern) ในโครงการนี้ Regex ถูกใช้ใน Phase 1 ของ Prepare Data Node เพื่อตรวจสอบว่าข้อความมี Prefix #แจ้งเคส หรือไม่ และสกัดข้อมูลสำคัญ เช่น รหัสสินค้า รหัสพนักงาน หมายเลขใบสั่ง ออกมาจากข้อความที่ผู้ใช้พิมพ์อย่างอิสระ (Mozilla Developer Network, 2568)

```
// =====
// PHASE 1: ตรวจสอบ #แจ้งเคส (REQUIRED) ถ้าไม่มี --- > skip : true ข้ามได้เลย ไม่มี prefix = bot อ่าน แต่ไม่เอามาเข้ากระบวนการ
// =====
const casePrefix = "#แจ้งเคส";
const hasCasePrefix = messageText.trim().startsWith(casePrefix);

if (!hasCasePrefix) {
  return [{ json: { skip: true, skip_reason: "ไม่มี #แจ้งเคส prefix", raw_message: messageText, replyToken: event.replyToken } }];
}
```

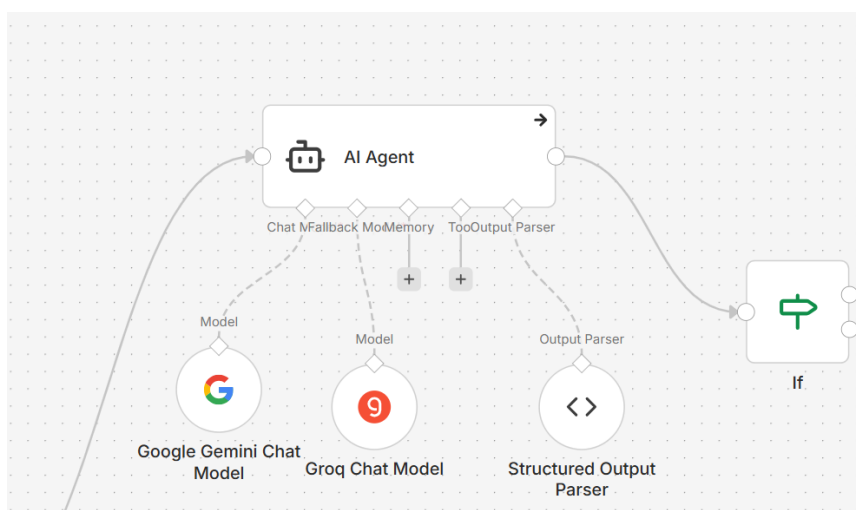
ภาพที่ 2. 3 code ตัวอย่างการ Regex หา Prefix #แจ้งเคส

2.4 Google Gemini AI และ Groq (AI Language Model)

โครงการนี้ใช้ Large Language Model (LLM) จำนวน 2 ตัวเป็น AI Engine ภายใน AI Agent Node

- Google Gemini: LLM จาก Google DeepMind มีความสามารถในการเข้าใจภาษาไทยและภาษาอังกฤษได้ดี ใช้เป็นโมเดลหลักในการวิเคราะห์ประเภทเคสและสกัดข้อมูลจากข้อความแชท
- Groq (Llama 3.3 70B): LLM ที่ขับเคลื่อนบน Hardware Groq LPU ซึ่งให้ความเร็วในการ Inference สูง ใช้เป็น Fallback Model เมื่อ Gemini ไม่สามารถตอบสนองได้

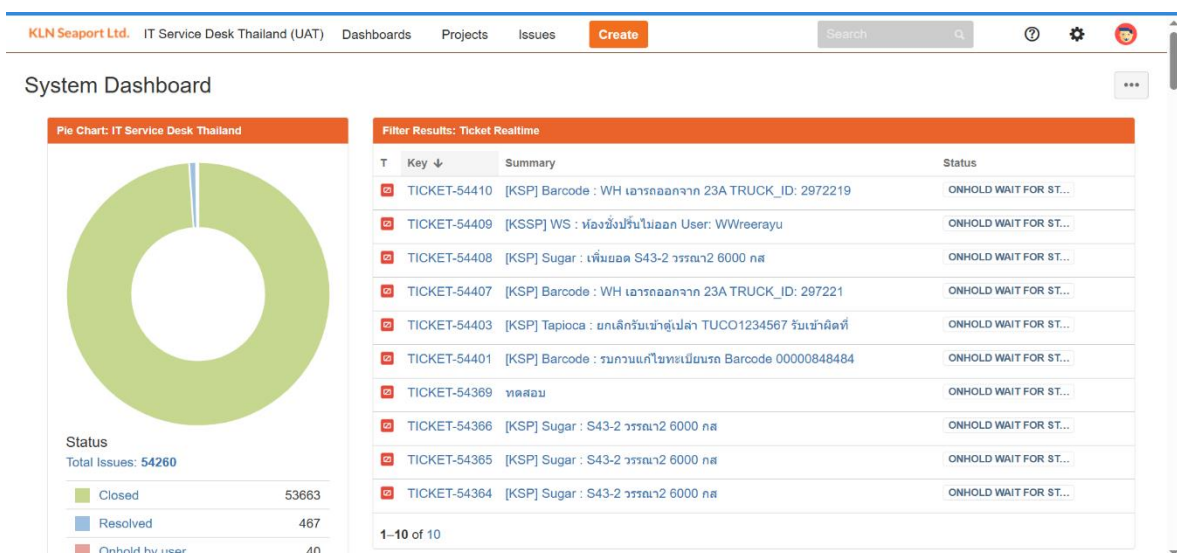
AI Agent จะรับ Prompt ที่กำหนด System Message ไว้ล่วงหน้า ให้ AI ทำหน้าที่วิเคราะห์ข้อความระบุประเภทเคส (case_type) ตรวจสอบความครบถ้วนของข้อมูล (is_complete) และสร้าง JSON Output ที่มีโครงสร้างมาตรฐาน เพื่อนำไปสร้าง Issue บน Jira ต่อไป (Google DeepMind, 2568)



ภาพที่ 2. 4 ตัวอย่าง AI Agent ที่ใช้ในการประมวลผลการแจ้งปัญหาของ User (เป็นเคสประเภทที่นอกเหนือของการ regex ข้อมูลไม่ครบ หรือ ขาดข้อมูลบางประเภท)

2.5 Jira Service Management

Jira เป็นระบบบริหารจัดการโปรเจกต์และ Issue Tracking ที่พัฒนาโดย Atlassian โดยในบริษัท KLN Seaport ใช้ Jira Service Management สำหรับบันทึกและติดตาม Incident ของทีม IT Service Desk ซึ่งในโครงการนี้ระบบจะสร้าง Issue บน Jira โดยอัตโนมัติเมื่อได้รับข้อมูลที่ครบถ้วน โดยกำหนดค่า Priority, Category, Summary และ Custom Fields ตามประเภทเคสที่วิเคราะห์ได้ (Atlassian, 2568)



ภาพที่ 2. 5 ตัวอย่าง System Dash board Jira Service Management

2.6 Google Sheets

Google Sheets ถูกนำมาใช้ในโครงการนี้ 2 บทบาทหลัก คือ (1) เป็น Storage สำหรับบันทึก Cache ของเคสที่ได้รับมาแล้ว เพื่อป้องกันการบันทึกข้อมูลซ้ำซ้อน (Duplicate Prevention) ภายในระยะเวลา 10 นาที และ (2) เป็น Data Log สำรองบันทึกข้อมูลทุกเคสที่ผ่านระบบสำหรับการตรวจสอบย้อนหลัง (n8n GmbH, 2568)

	A	B	C	D	E	F	G
1	cache_key	created_at					
2	U17cd24b19cc1e44f130256d23d323_ขมยุดเครื่องขึ้นใหม่และ: บันไดออก	2026-03-27T08:26:55.553Z					
3	U17cd24b19cc1e44f130256d23d323_ขมยุดเครื่องขึ้นใหม่และ: บันไดออก ที่ห้อง wweerayu	2026-03-27T08:36:22.822Z					
4	U17cd24b19cc1e44f130256d23d323_ขมยุดเครื่องขึ้นใหม่และ: บันไดออก ที่ห้อง wweerayu	2026-03-27T09:08:18.995Z					
5	U17cd24b19cc1e44f130256d23d323_user: wweerayu เครื่องขึ้นใหม่และ: บันไดออก	2026-03-27T09:08:49.450Z					
6	U17cd24b19cc1e44f130256d23d323_ขมยุดเครื่องขึ้นใหม่และ: บันไดออก	2026-03-27T09:08:49.450Z					

ภาพที่ 2. 6 ตัวอย่าง Google Sheet ที่ใช้ในการเก็บ log ของ ปัญหาที่ User แจ้งเข้ามาเพื่อแก้ปัญหาการแจ้งเคสซ้ำ (หากมีข้อความเดิมภายใน 10 ที่ และมาจาก User คนเดิมจะไม่สาธตแจ้งซ้ำได้)

	A	B	C	D
1	reporter	raw_message	source_group	received_at
2	U17cdf24b19cc1e44f130256df23d3f23	543-2 วรรณ 2 นียอด 6,000 กส	U17cdf24b19cc1e44f130256df23d3f23	2026-02-19T02:49:59.962Z
3	U17cdf24b19cc1e44f130256df23d3f23	543-2 วรรณ 2 นียอด 6,000 กส	C3d805526a7466a5385d5aa7b6e3fa32a	2026-02-19T03:03:14.295Z
4	U17cdf24b19cc1e44f130256df23d3f23	543-2 วรรณ 2 นียอด 6,000 กส	U17cdf24b19cc1e44f130256df23d3f23	2026-02-19T03:35:29.633Z
5	U17cdf24b19cc1e44f130256df23d3f23	543-2 วรรณ 2 นียอด 6,000 กส	C3d805526a7466a5385d5aa7b6e3fa32a	2026-02-19T03:35:54.621Z
6	U17cdf24b19cc1e44f130256df23d3f23	543-2 วรรณ 2 นียอด 6,000 กส	C3d805526a7466a5385d5aa7b6e3fa32a	2026-02-19T04:09:54.117Z
7	U17cdf24b19cc1e44f130256df23d3f23	543-2 วรรณ 2 นียอด 6,000 กส	U17cdf24b19cc1e44f130256df23d3f23	2026-02-19T04:10:47.101Z
8	U17cdf24b19cc1e44f130256df23d3f23	543-2 วรรณ 2 นียอด 6,000 กส	U17cdf24b19cc1e44f130256df23d3f23	2026-02-19T04:31:42.729Z
9	U17cdf24b19cc1e44f130256df23d3f23	543-2 วรรณ 2 นียอด 6,000 กส	U17cdf24b19cc1e44f130256df23d3f23	2026-02-19T07:08:11.378Z
10	U17cdf24b19cc1e44f130256df23d3f23	543-2 วรรณ 2 นียอด 6,000 กส	C3d805526a7466a5385d5aa7b6e3fa32a	2026-02-19T07:33:06.138Z
11	U17cdf24b19cc1e44f130256df23d3f23	543-2 วรรณ 2 นียอด 6,000 กส	U17cdf24b19cc1e44f130256df23d3f23	2026-02-19T07:41:01.495Z
12	U17cdf24b19cc1e44f130256df23d3f23	543-2 วรรณ 2 นียอด 6,000 กส	C3d805526a7466a5385d5aa7b6e3fa32a	2026-02-19T08:12:58.448Z
13	U17cdf24b19cc1e44f130256df23d3f23	543-2 วรรณ 2 นียอด 6,000 กส	U17cdf24b19cc1e44f130256df23d3f23	2026-02-19T08:17:40.075Z
14	U17cdf24b19cc1e44f130256df23d3f23	543-2 วรรณ 2 นียอด 6,000 กส	C3d805526a7466a5385d5aa7b6e3fa32a	2026-02-23T04:42:01.356Z
15	U17cdf24b19cc1e44f130256df23d3f23	543-2 วรรณ 2 นียอด 6,000 กส	C3d805526a7466a5385d5aa7b6e3fa32a	2026-02-23T04:42:41.776Z
16	U17cdf24b19cc1e44f130256df23d3f23	543-2 วรรณ 2 นียอด 6,000 กส	C3d805526a7466a5385d5aa7b6e3fa32a	2026-02-23T04:55:53.917Z
17	U17cdf24b19cc1e44f130256df23d3f23	543-2 วรรณ 2 นียอด 6,000 กส	C3d805526a7466a5385d5aa7b6e3fa32a	2026-02-23T04:55:53.917Z

ภาพที่ 2. 7 ตัวอย่าง Google sheet ที่เป็น Storage เพื่อเก็บการสร้างตัว Ticket แล้ว

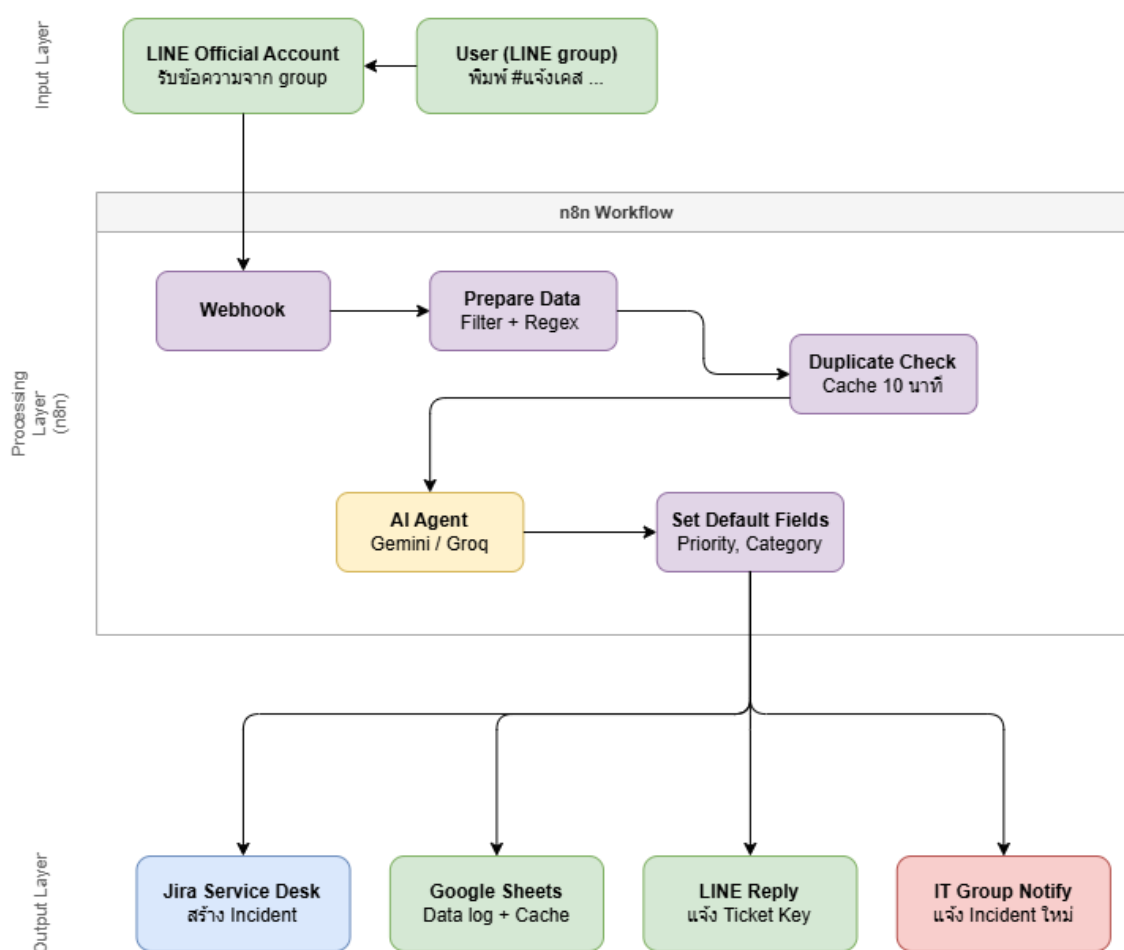
บทที่ 3

การออกแบบและพัฒนาระบบ

3.1 สถาปัตยกรรมระบบ (System Architecture)

ระบบ LINE-2-JIRA ออกแบบในรูปแบบ Event-driven Architecture โดยมีการทำงานเป็นลำดับขั้นดังนี้:

- 3.1.1 **ขั้นที่ 1** - Input Layer: LINE Official Account รับข้อความจากผู้ใช้งานในกลุ่ม LINE แล้วส่งผ่าน Webhook ไปยัง n8n
- 3.1.2 **ขั้นที่ 2** - Processing Layer: n8n Workflow ประมวลผลข้อมูลผ่านขั้นตอนต่าง ๆ ซึ่งประกอบด้วย การตรวจสอบ Prefix, การ Filter เหตุการณ์, การป้องกัน Duplicate, การวิเคราะห์ด้วย AI
- 3.1.3 **ขั้นที่ 3** - Output Layer: บันทึกข้อมูลลง Google Sheets, สร้าง Jira Issue, และส่ง Reply กลับไปยังผู้แจ้งผ่าน LINE



ภาพที่ 2. 8 ตัวอย่างโครงสร้างระบบ

3.2 การออกแบบ Workflow

Workflow ทั้งหมดในโครงการนี้ถูกพัฒนามาบน n8n และประกอบด้วย 33 Node โดยแบ่งการทำงานออกเป็น 6 ขั้นตอนหลัก ดังนี้:

3.2.1 ขั้นตอนที่ 1: รับและกรองข้อมูลเบื้องต้น (Prepare Data Node)

เมื่อมีข้อความส่งเข้ามาจาก LINE Webhook ระบบจะเริ่มด้วยการประมวลผลใน Prepare Data Node ซึ่งทำหน้าที่:

- Phase 0: ตรวจสอบประเภทเหตุการณ์ — รับเฉพาะ Event ที่เป็น message type และ text type เท่านั้น เพื่อกรอง Event ที่ไม่เกี่ยวข้อง เช่น การเข้าร่วมกลุ่ม หรือการส่งรูปภาพ
- Phase 1: ตรวจสอบ Prefix #แจ้งเคส — ข้อความต้องเริ่มต้นด้วย #แจ้งเคส มิฉะนั้นระบบจะส่ง skip: true และไม่ประมวลผลต่อ
- Phase 2: Normalize ข้อความ — ลบอักขระพิเศษ, ช่องว่างซ้ำ และทำ Unicode NFC Normalization เพื่อความสม่ำเสมอของข้อมูล
- Phase 3: สร้าง Metadata — บันทึก userId, groupId, displayName, timestamp และ raw_message เพื่อส่งต่อไปยังโหนดถัดไป

3.2.2 ขั้นตอนที่ 2: ป้องกันข้อมูลซ้ำซ้อน (Duplicate Prevention)

ระบบมีกลไกป้องกัน Duplicate โดยใช้ Google Sheets เป็น Cache Store ทำงานผ่าน 3 Node ได้แก่ Read Duplicate Log, Check Duplicate1 และ Write Duplicate Log

หลักการทำงานคือสร้าง Unique Key จาก ชื่อผู้แจ้ง (reporter) + เนื้อหาข้อความ 80 ตัวแรก แล้วตรวจสอบว่ามีรายการที่มี Key เดียวกันถูกบันทึกไว้ภายใน 10 นาทีที่ผ่านมาหรือไม่ ถ้าพบว่าเป็น Duplicate ระบบจะส่ง Reply แจ้งผู้ใช้งานและหยุดประมวลผล

3.2.3 ขั้นตอนที่ 3: วิเคราะห์เคสด้วย AI (AI Agent Node)

ข้อมูลที่ผ่านการ Filter แล้วจะถูกส่งไปยัง AI Agent Node ซึ่งใช้ Google Gemini เป็น LLM หลัก และ Groq (Llama 3.3 70B) เป็น Fallback โดย AI จะได้รับ System Prompt ที่กำหนดให้ทำหน้าที่เป็นระบบวิเคราะห์เคสแจ้งปัญหาของระบบโลจิสติกส์

AI จะประมวลผลและส่งคืน JSON ที่มีโครงสร้างดังนี้:

- is_case: ระบุว่าข้อความนี้เป็นการแจ้งเคสหรือไม่ (Boolean)
- case_type: ประเภทเคส เช่น SUGAR, WS, WH, COIL, PASSWORD, KIOSK เป็นต้น
- is_complete: ข้อมูลครบถ้วนหรือไม่ (Boolean)
- missing_fields: รายการ Field ที่ยังขาด (Array)
- extracted_fields: ข้อมูลที่สกัดได้ เช่น product_code, warehouse, quantity, truck_id เป็นต้น
- reply_message: ข้อความตอบกลับที่เหมาะสมสำหรับส่งกลับไปยังผู้แจ้ง

3.2.4 ขั้นตอนที่ 4: กำหนดค่า Default Fields (Set Default Fields Node)

หลังจาก AI วิเคราะห์เคสแล้ว Set Default Filed Node จะแปลง case_type ไปเป็นค่า Jira Fields โดยใช้ Mapping Table ดังต่อไปนี้:

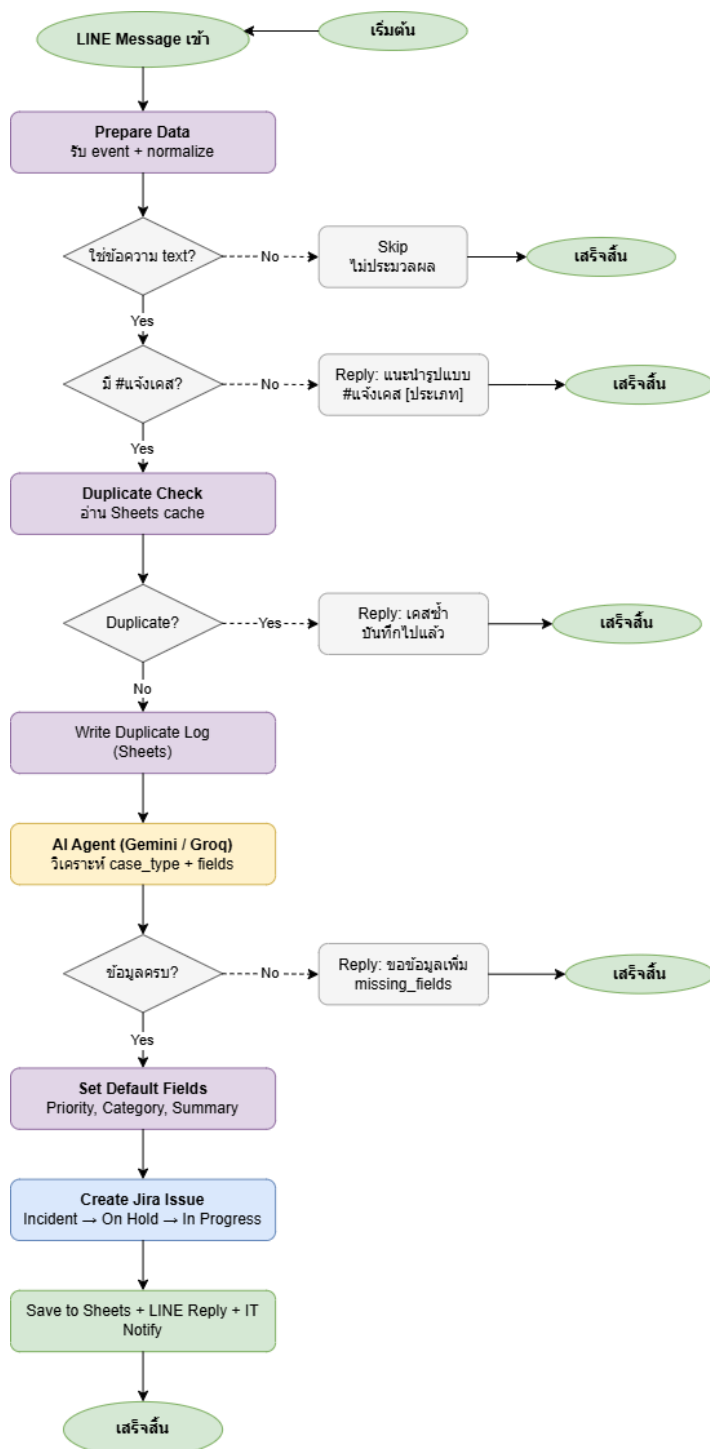
- categoryMap: แปลง case_type เป็น Jira Category เช่น SUGAR → General Support, WS/WH/COIL → KWMS, WOODCHIP/TAPIOCA → KPWMS
- priorityMap: กำหนด Priority เช่น SUGAR/WS/WH/COIL/GC/BARCODE → 1 Critical, KIOSK → 2 Important, PASSWORD/FREEZONE/USER_IT → 3 Standard
- buildSummary: สร้าง Summary อัตโนมัติจาก case_type และ extracted_fields เพื่อใช้เป็นชื่อ Jira Issue

3.2.5 ขั้นตอนที่ 5: สร้าง Issue บน Jira (Create an Issue Node)

เมื่อข้อมูลครบถ้วน ระบบจะสร้าง Jira Issue ใน Project TEST: IT Service Desk Thailand โดยอัตโนมัติ โดยกำหนด Issue Type เป็น Incident และกรอกข้อมูลต่าง ๆ ตามที่ AI วิเคราะห์ได้ จากนั้น Issue จะถูก Transition ผ่านสถานะ On Hold → In Progress โดยอัตโนมัติ

3.2.6 ขั้นตอนที่ 6: ตอบกลับและแจ้งเตือน (Reply & Notify)

ระบบจะส่ง Reply กลับไปยังกลุ่ม LINE ที่แจ้งเคสมา พร้อมสรุปข้อมูลเคสและ Jira Ticket Key เพื่อให้ผู้แจ้งรับทราบ นอกจากนี้ยังส่ง Notification ไปยังกลุ่ม IT Group แยกต่างหาก เพื่อแจ้งให้ทีม IT ทราบว่ามี Incident ใหม่เข้ามา

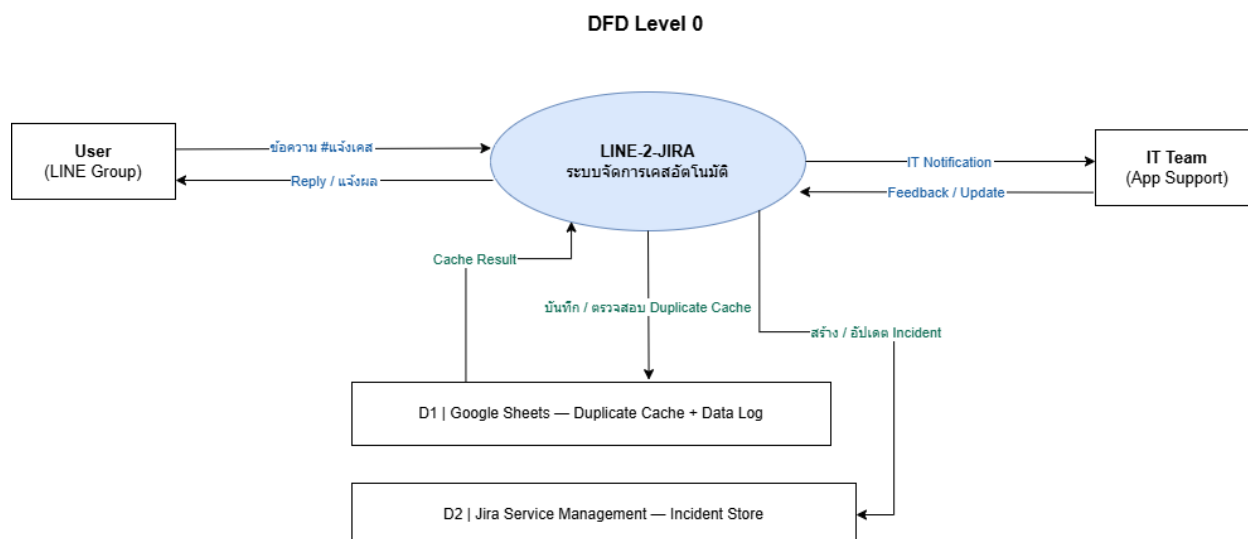


ภาพที่ 2. 9 Workflow Diagram

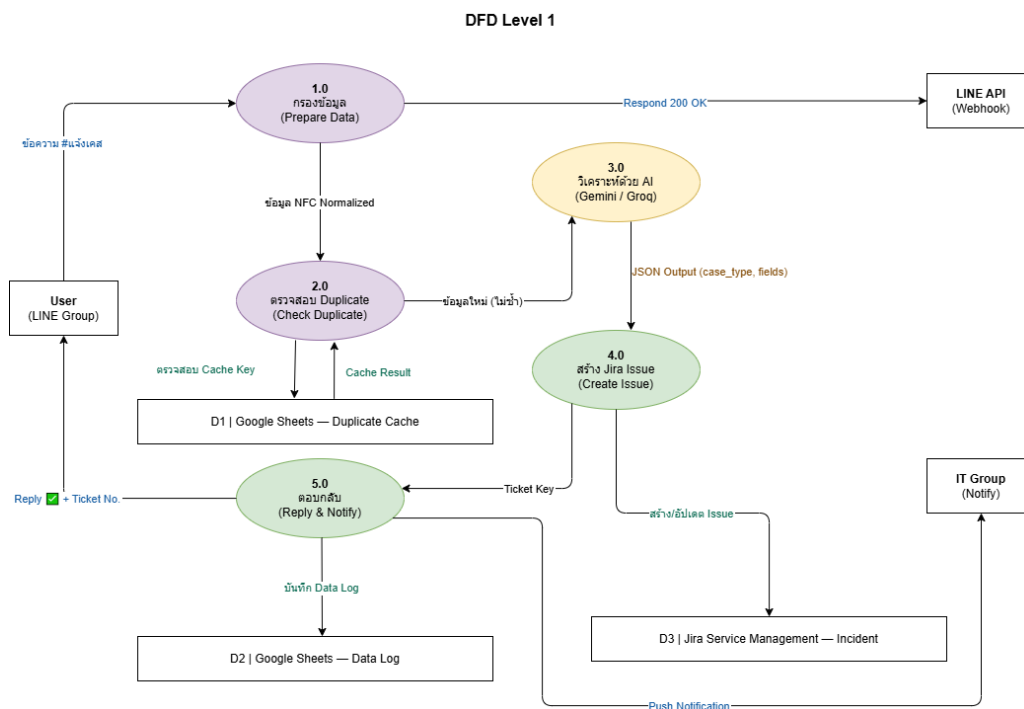
3.3 การออกแบบประเภทเคส (Case Type Design)

ระบบรองรับ 12 ประเภทเคส ซึ่งออกแบบมาให้ครอบคลุมการแจ้งปัญหาทุกประเภทในบริษัท KLN Seaport ดังนี้:

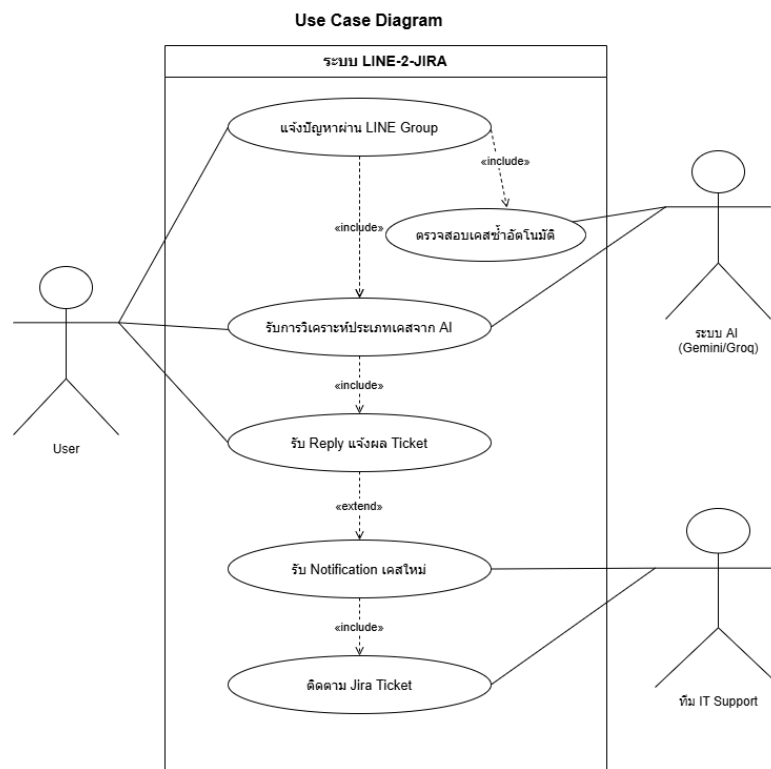
- SUGAR (น้ำตาล): สำหรับปัญหาที่เกี่ยวกับการจัดเก็บและเบิกจ่ายน้ำตาล ต้องการข้อมูล: รหัสสินค้า, ชื่อคลัง, จำนวน, หน่วย และ Action เช่น เพิ่มยอด, ตั้บบรรจุ, ยอดเบิ้ล, ยกเลิก, แก้ไข
- WS (Weight Scale - ห้างชั่ง): ปัญหาเกี่ยวกับระบบชั่งน้ำหนัก ต้องการ: เลขใบชั่ง (scale_no) หรือรหัสสินค้า และ Action เช่น ตีกลับ, ยกเลิก
- WH (Warehouse - โกดังเหล็ก): ปัญหาเกี่ยวกับการจัดการสินค้าในโกดัง ต้องการ: TRUCK_ID หรือ Barcode และ Action
- COIL (Steel Coil): ปัญหาเกี่ยวกับ Steel Coil ต้องการ: Barcode หรือ Coil ID และ Action
- WOODCHIP (ไม้สับ): ปัญหาเกี่ยวกับระบบจัดการไม้สับ ต้องการ: เลขใบชั่งหรือ TRUCK_ID
- TAPIOCA (มันสำปะหลัง): ปัญหาเกี่ยวกับมันสำปะหลัง ต้องการ: LOT หรือ Barcode
- FREEZONE: ปัญหาเกี่ยวกับ Free Zone ต้องการ: Barcode และ Action
- GC (General Cargo): สินค้าทั่วไป ต้องการ: Document, Vessel หรือ Coil ID
- PASSWORD: การรีเซ็ตหรือปลดล็อก Password ต้องการ: Username (AD) และ System เช่น AD, O365, VPN, KWMS
- KIOSK: ปัญหาเครื่อง Kiosk ต้องการ: ตำแหน่งประตู (gate_location) และอาการเสีย (symptom)
- USER_IT (IT Support): ปัญหา IT ทั่วไป ต้องการ: Username หรือ System และ Action
- BARCODE: ปัญหา Barcode ต้องการ: TRUCK_ID หรือ Barcode Number



ภาพที่ 2. 10 ตัวอย่าง Data Flow Level 0



ภาพที่ 2. 11 ตัวอย่าง Data Flow Diagram Level 1



ภาพที่ 2. 12 ตัวอย่าง Use Case Flow

บทที่ 4

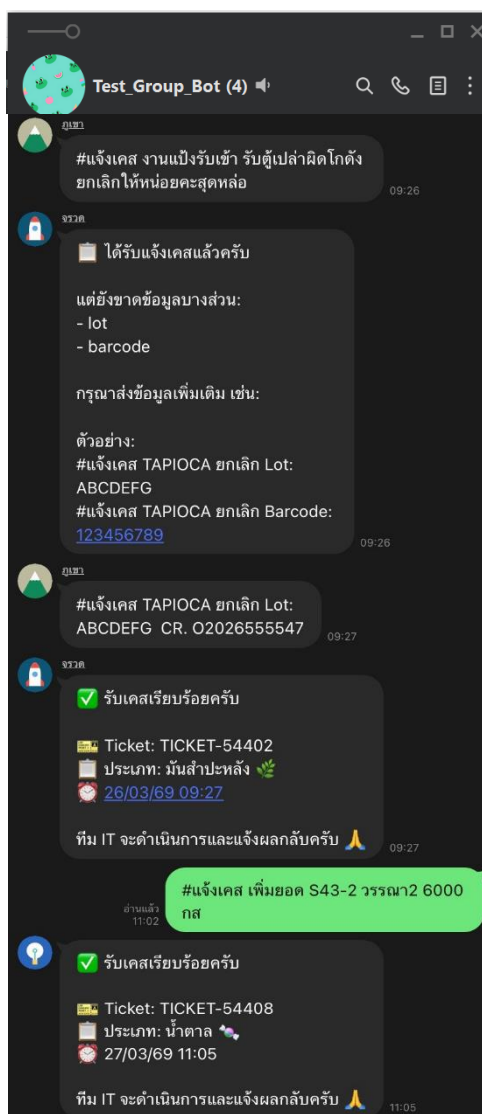
ผลการดำเนินงาน

4.1 ผลลัพธ์ที่ได้จากระบบ

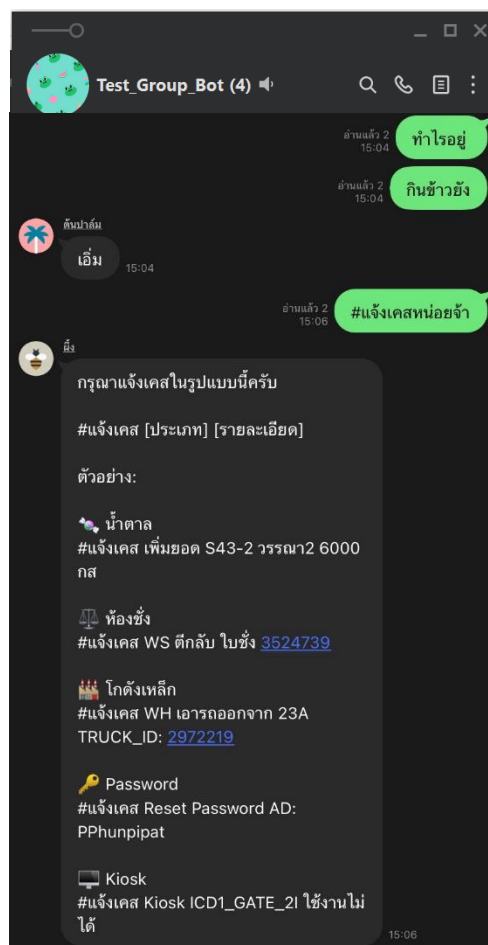
ระบบ LINE-2-JIRA ได้รับการพัฒนาและทดสอบในสภาพแวดล้อม UAT (User Acceptance Test) โดยมีผลลัพธ์ที่สำคัญดังนี้:

4.1.1 ความสามารถในการรับและกรองข้อมูลอัตโนมัติ

ระบบสามารถรับข้อความจาก LINE Group ได้แบบ Real-time ผ่าน Webhook และกรองเฉพาะข้อความที่มี Prefix #แจ้งเคส โดยอัตโนมัติ ข้อความที่ไม่เกี่ยวข้อง เช่น การพูดคุยทั่วไป การส่งรูปภาพ และ Sticker จะถูกกรองออกโดยไม่มีการประมวลผลเพิ่มเติม ส่งผลให้ระบบมีประสิทธิภาพในการใช้งาน Resource



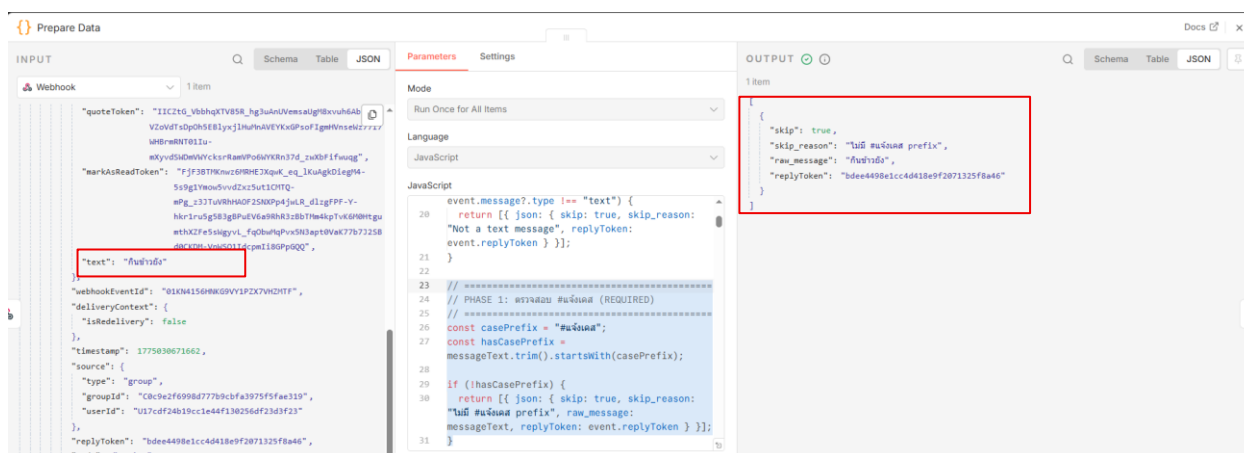
ภาพที่ 2. 13 ตัวอย่างการรับข้อความจาก Line Group และการ Reply



ภาพที่ 2. 14 ตัวอย่างการพิมพ์ข้อความอื่นที่ไม่ใช่ #แจ้งเคส



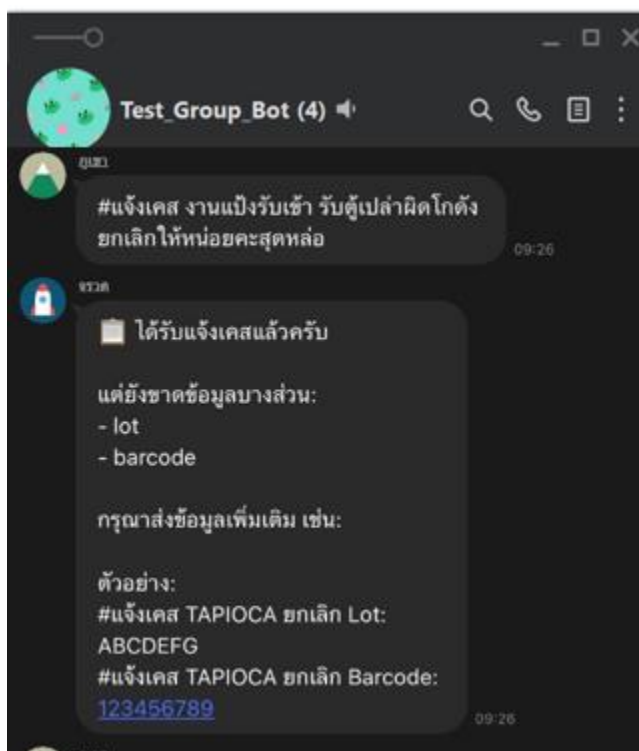
ภาพที่ 2. 15 ตัวอย่างการทำงานของระบบของข้อความที่ไม่ใช่ #แจ้งเคส



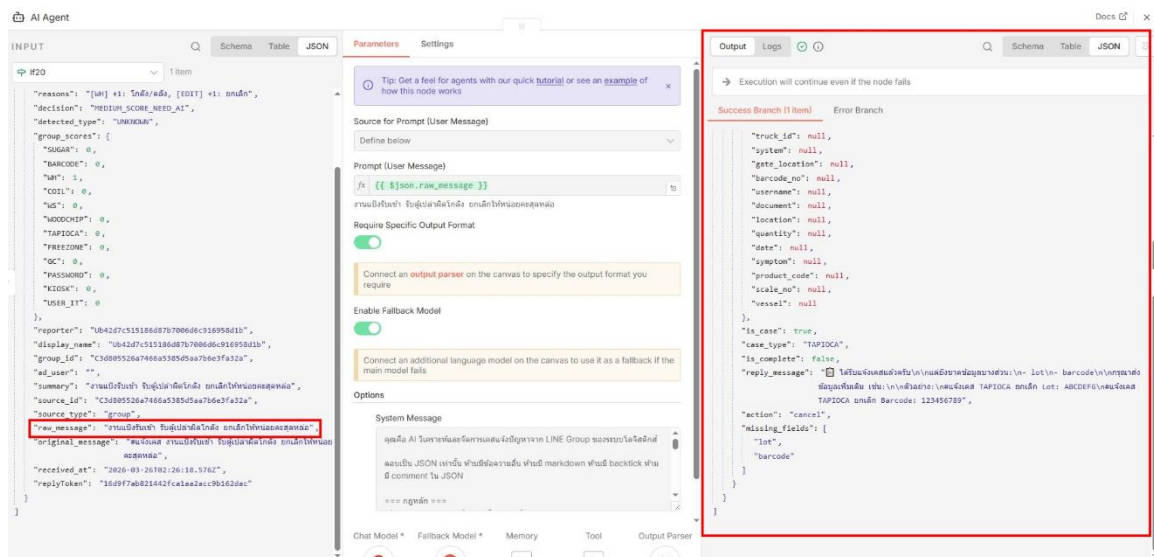
ภาพที่ 2. 16 ตัวอย่าง การทำงานตรวจสอบข้อความที่ไม่มี #แฮชแท็ก

4.1.2 ความแม่นยำในการจำแนกประเภทเคส

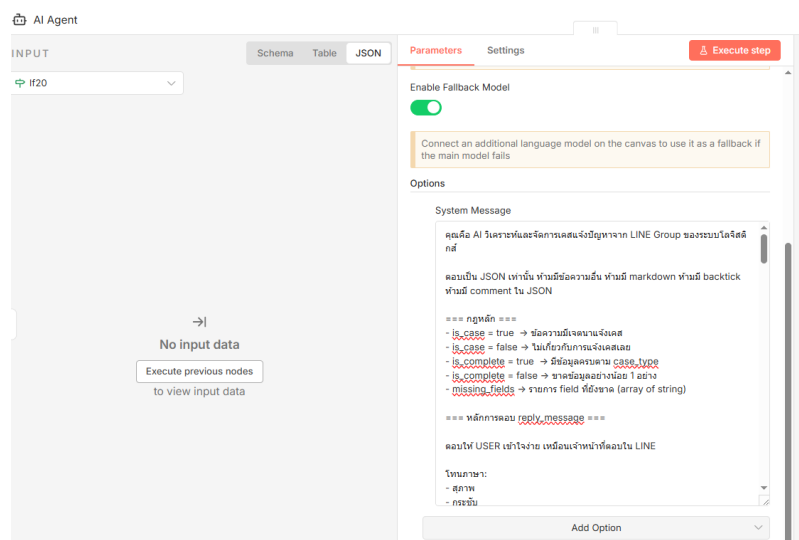
AI Agent ที่ขับเคลื่อนด้วย Google Gemini สามารถจำแนกประเภทเคสทั้ง 12 ประเภทได้อย่างแม่นยำ โดยเฉพาะเคสที่มีรูปแบบข้อความชัดเจน เช่น น้ำตาล, ห้องซั้ว และ Password นอกจากนี้ AI ยังสามารถระบุ Field ที่ขาดหายไปและสื่อสารกลับไปยังผู้แจ้งในภาษาที่เข้าใจง่ายได้



ภาพที่ 2. 17 ตัวอย่างการตอบกลับของ BOT กรณีรับปัญหาแต่ยังขาดข้อมูล



ภาพที่ 2. 18 ตัวอย่างการทำงานของบริการตอบกลับ User



ภาพที่ 2. 19 ตัวอย่าง Node Ai Agent

คุณคือ AI วิเคราะห์และจัดการเคสแจ้งปัญหาจาก LINE Group ของระบบโลจิสติกส์

ตอบเป็น JSON เท่านั้น ห้ามมีข้อความอื่น ห้ามมี markdown ห้ามมี backtick ห้ามมี comment ใน JSON

=== กฎหลัก ===

- is_case = true → ข้อความมีเจตนาแจ้งเคส
- is_case = false → ไม่เกี่ยวกับการแจ้งเคสเลย
- is_complete = true → มีข้อมูลครบตาม case_type
- is_complete = false → ขาดข้อมูลอย่างน้อย 1 อย่าง
- missing_fields → รายการ field ที่ยังขาด (array of string)

=== หลักการตอบ reply_message ===

ตอบให้ USER เข้าใจง่าย เหมือนเจ้าหน้าที่ตอบใน LINE โทนภาษา:

- สุภาพ
- กระชับ
- ใช้ emoji ได้เล็กน้อย
- บอกสิ่งที่ยังขาดให้ชัด

=== รูปแบบการตอบ ===

กรณี 1: ไม่ใช่การแจ้งเคส (is_case = false) reply_message:

กรุณาแจ้งเคสในรูปแบบนี้ครับ

#แจ้งเคส [ประเภท] [รายละเอียด]

ตัวอย่าง:



น้ำตาล

#แจ้งเคส เพิ่มยอด S43-2 วรรณา2 6000 กส



ห้องซัง #แจ้งเคส WS ตีกลับ ใบซัง 3524739



โกดังเหล็ก #แจ้งเคส WH เอารถออกจาก 23A TRUCK_ID: 2972219



Password

#แจ้งเคส Reset Password AD: PPhunpipat



Kiosk

#แจ้งเคส Kiosk ICD1_GATE_2I ใช้งานไม่ได้

กรณี 2: เป็นเคส แต่ข้อมูลไม่ครบ (is_complete = false)

ตอบแบบนี้: 📄 ได้รับแจ้งเคสแล้วครับ แต่ยังขาดข้อมูลบางส่วน: - [missing_fields] กรุณาส่งข้อมูลเพิ่มเติม

เช่น: ตัวอย่าง: (สร้างตัวอย่างตาม field ที่ขาด) ตัวอย่างข้อความ:

#แจ้งเคส Reset Password AD: PPhunpipat

กรณี 3: ข้อมูลครบ (is_complete = true) ตอบแบบนี้:



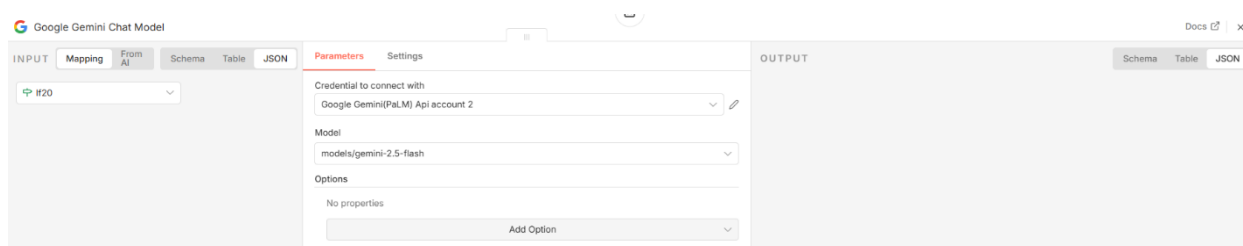
รับเคสเรียบร้อยแล้วครับ



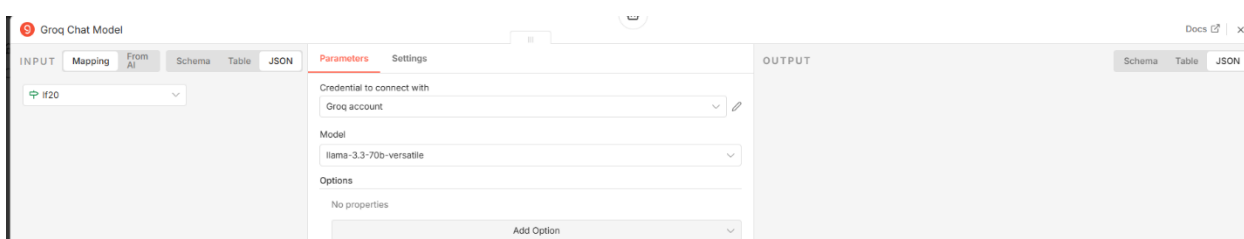
ประเภทเคส: [case_type] ข้อมูลที่ได้รับ: - action - field ที่มีค่า

ทีมงานกำลังดำเนินการให้ครับ

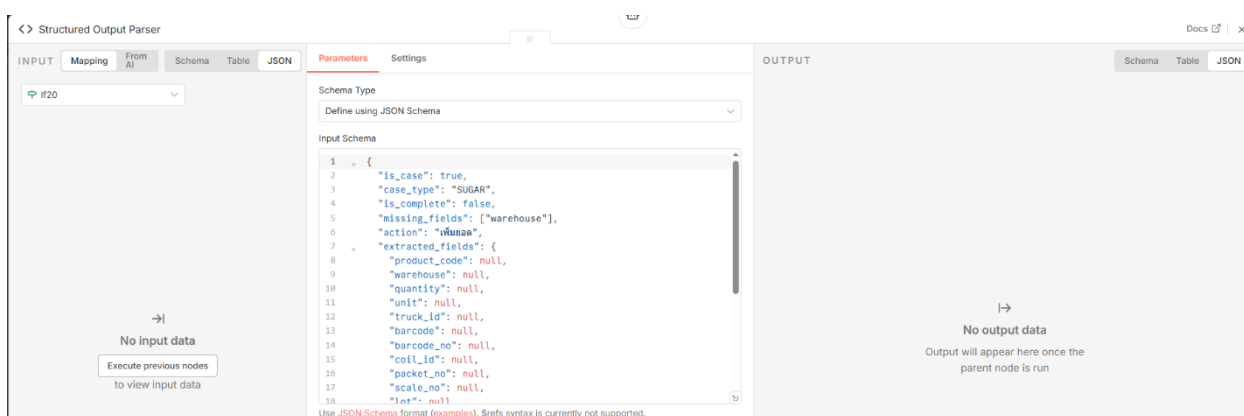
ภาพที่ 2. 20 ตัวอย่าง Prompt Ai



ภาพที่ 2. 21 ตัวอย่าง Model Google Gemini



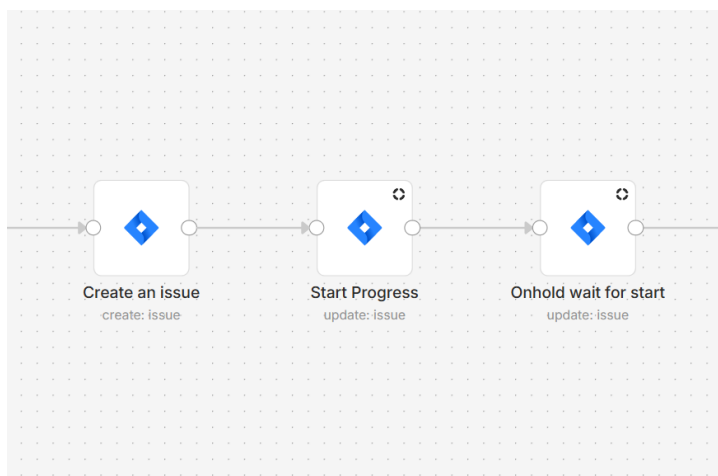
ภาพที่ 2. 22 ตัวอย่าง Model Groq



ภาพที่ 2. 23 ตัวอย่าง Structured Output

4.1.3 การสร้าง Jira Issue อัตโนมัติ

เมื่อข้อมูลครบถ้วน ระบบสามารถสร้าง Jira Issue ใน IT Service Desk ได้โดยอัตโนมัติ โดย Issue จะมีข้อมูลครบถ้วนทั้ง Summary, Priority, Category และ Custom Fields ตามประเภทเคสที่กำหนด



ภาพที่ 2. 24 ตัวอย่าง Node การสร้าง Ticket

INPUT

```

{
  "Impact": {
    "value": "3 Effects one of few user"
  },
  "urgency": {
    "value": "3 Standard"
  },
  "description": "ICD1_GATE_21 ไม่ผ่านใบไม้",
  "summary": "[KSSP] Kiosk : ICD1_GATE_21 ไม่ผ่านใบไม้",
  "received_at": "2026-04-01T08:37:00.570Z",
  "raw_message": "ICD1_GATE_21 ไม่ผ่านใบไม้",
  "source_group": "C3d98552a7466a5385d5aa7b6e3fa32a",
  "issue_type": "Incident",
  "reporter": "Uadafa21d85e8cd07bc3a52700ec1389",
  "project": "IT Service Desk Thailand (Ticket)",
  "mode": {
    "value": "Web Form"
  },
  "site": {
    "value": "KSSP-Sriracha"
  },
  "category": {
    "value": "General Support"
  },
  "technician": {
    "value": "KSSP- App Support"
  }
}
  
```

Parameters

Credential to connect with: Jira SW Cloud account

Resource: Issue

Operation: Create

Project: From list TEST: IT Service Desk Thailand

Issue Type: From list Incident

Summary: `fx {{ $json.summary }}`

Additional Fields

Custom Fields

Field: By ID customfield_30008

Field Value: `fx {{ $json.mode }}`

[Object: {"value": "Web Form"}]

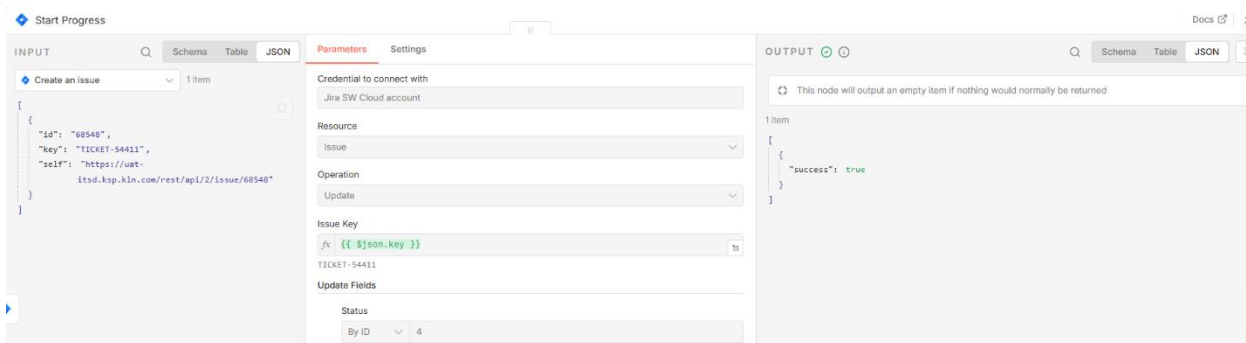
Field: By ID customfield_30011

OUTPUT

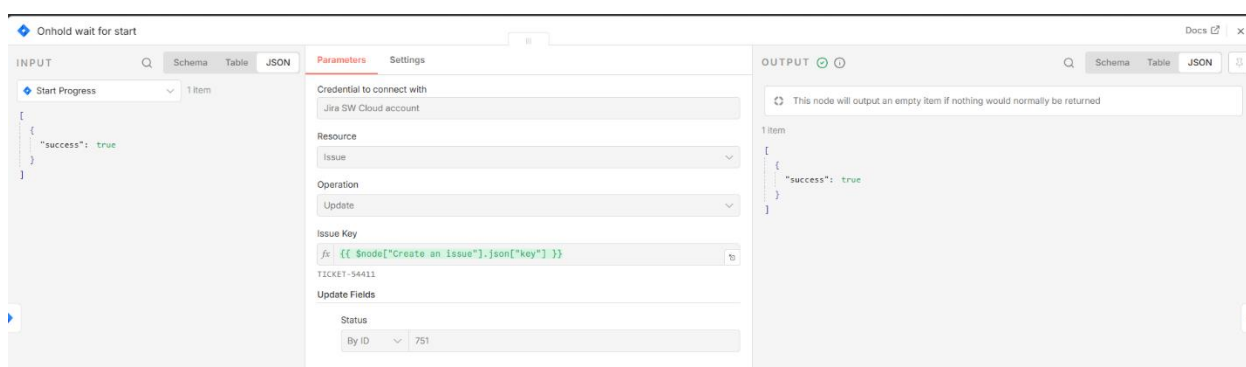
```

1 item
{
  "id": "68548",
  "key": "TICKET-54431",
  "self": "https://uat-itsd.ksp.kln.com/rest/api/2/issue/68548"
}
  
```

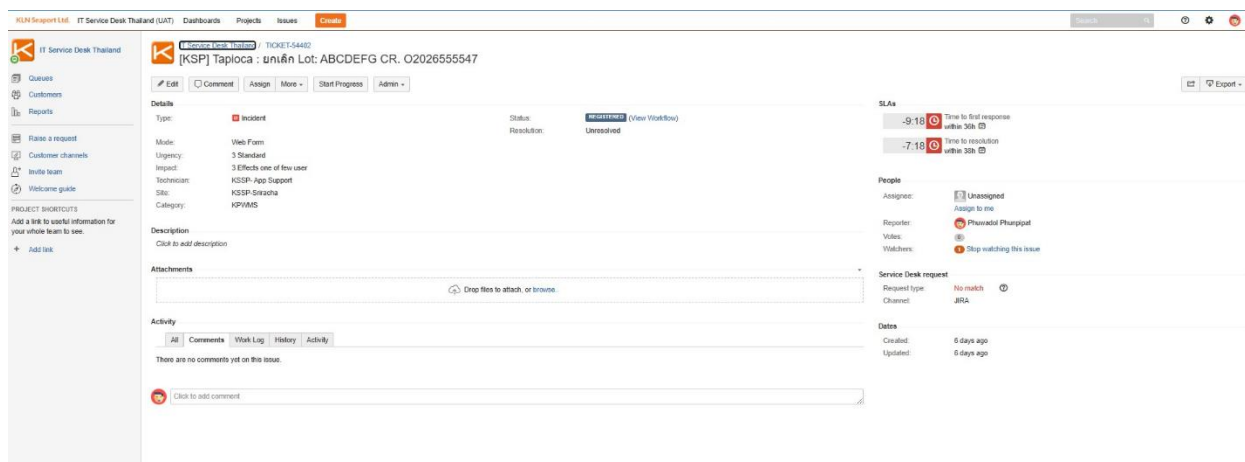
ภาพที่ 2. 25 ตัวอย่าง การสร้าง Ticket



ภาพที่ 2. 26 ตัวอย่างการ เริ่ม Start Process



ภาพที่ 2. 27 ตัวอย่าง การเปลี่ยน Status ของ Ticket ให้อยู่ในการรอผู้รับผิดชอบปัญหานี้เข้ามา Start Process ด้วยตัวเอง



ภาพที่ 2. 28 ตัวอย่างการเปิด Ticket ใน Jira สำเร็จ

Edit Issue : TICKET-54402 Configure Fields

Issue Type* Incident

Reporter Phuwadol Phunpipat

Mode* Web Form

Urgency 3 Standard

Impact 3 Effects one of few user

Technician* KSSP-App Support

Assignee Unassigned

Site* KSSP-Sriracha

Category* KPWMS None

Summary* [KSP] Tapioca : ยกเลิก Lot: ABCDEFG CR. O2026555547

Description

Attachment Drop files to attach, or browse.

Comment Respond to customer Internal comment

Your comment will be visible to customers. Embed attachments to make them visible to customers.

Update Cancel

ภาพที่ 2. 29 ตัวอย่างข้อมูลที่รับมาจาก N8N ก็จะบันทึกตาม หัวข้อที่รับข้อความมาจาก User

4.2 ปัญหาและการแก้ไขระหว่างการพัฒนา

ในระหว่างกระบวนการพัฒนาและทดสอบระบบ พบปัญหาและแนวทางแก้ไขที่สำคัญดังนี้:

- 4.2.1 ปัญหา Unicode และการ Encode ข้อความภาษาไทย ข้อความภาษาไทยที่ส่งมาจาก LINE บางครั้งมี Unicode Non-Breaking Space (\u00a0) แทรกอยู่ ทำให้ Regex ไม่สามารถ Match รูปแบบที่กำหนดได้ แก้ไขโดยการทำ normalize("NFC") และ replace เพื่อลบอักขระพิเศษออกก่อนประมวลผล
- 4.2.2 ปัญหาการวิเคราะห์ที่คลุมเครือจาก AI
ในบางกรณี AI วิเคราะห์ประเภทเคสได้ไม่ชัดเจน เนื่องจากข้อความของผู้ใช้งานไม่ระบุประเภทสินค้าหรือระบบที่เกี่ยวข้องชัดเจน แก้ไขโดยการปรับปรุง System Prompt ให้มีตัวอย่างข้อความและกฎเพิ่มเติม รวมถึงกำหนดให้ใช้ USER_IT เป็น Default เมื่อไม่สามารถระบุประเภทได้
- 4.2.3 ปัญหา Timeout ของ AI Model บางครั้ง Google Gemini ใช้เวลาตอบสนองนานเกินไป ส่งผลให้ Workflow หมดเวลา แก้ไขโดยเพิ่ม Groq (Llama 3.3 70B) เป็น Fallback Model ซึ่งมี Inference

4.3 การทดสอบระบบ

การทดสอบระบบดำเนินการใน 3 รูปแบบ ได้แก่:

- 4.3.1 Unit Testing: ทดสอบแต่ละ Node แยกกัน โดยใช้ Pin Data ใน n8n เพื่อกำหนดข้อมูล Input คงที่ และตรวจสอบ Output ของแต่ละ Node Integration Testing: ทดสอบ Workflow ทั้งหมดแบบ End-to-End โดยส่งข้อความจาก LINE Group จริงในสภาพแวดล้อม UAT
- 4.3.2 Integration Testing: ทดสอบ Workflow ทั้งหมดแบบ End-to-End โดยส่งข้อความจาก LINE Group จริงในสภาพแวดล้อม UAT
- 4.3.3 User Acceptance Testing (UAT): ให้เจ้าหน้าที่จาก App Support Team ทดสอบระบบจริงและให้ Feedback เพื่อปรับปรุง Logic และ Reply Message ให้สอดคล้องกับการทำงานหน้างาน

บทที่ 5

สรุปและข้อเสนอแนะ

5.1 สรุปผลการดำเนินงาน

โครงการ LINE-2-JIRA: ระบบจัดเก็บข้อมูลแจ้งปัญหาอัตโนมัติสำหรับทีม App Support บรรลุวัตถุประสงค์หลักทั้ง 4 ข้อที่กำหนดไว้ ได้แก่

- (1) พัฒนาระบบรับแจ้งปัญหาผ่าน LINE Official Account ที่สามารถคัดกรองข้อมูลได้อัตโนมัติ
- (2) จัดรูปแบบข้อมูลแชทที่กระจัดกระจายให้เป็น Structured Data
- (3) บันทึกข้อมูลลงระบบโดยอัตโนมัติแทนการบันทึกด้วยมือ และ
- (4) เพิ่มความเร็วและลดความผิดพลาดในการรวบรวมข้อมูล ระบบที่พัฒนาขึ้นสามารถทำงานได้อย่างครบวงจร ตั้งแต่การรับข้อความจาก LINE ผ่านการวิเคราะห์ด้วย AI จนถึงการสร้าง Jira Issue และแจ้งเตือนทีม IT โดยอัตโนมัติ ลดภาระงาน Manual ของเจ้าหน้าที่อย่างมีนัยสำคัญ และช่วยให้ข้อมูลการแจ้งปัญหามีมาตรฐานและครบถ้วนยิ่งขึ้น

5.2 ประสบการณ์และความรู้ที่ได้รับ

จากการดำเนินโครงการสหกิจศึกษาครั้งนี้ ผู้จัดทำได้รับความรู้และประสบการณ์ที่มีคุณค่าในหลายด้าน ได้แก่ การออกแบบและพัฒนา Workflow Automation โดยใช้ n8n ในสภาพแวดล้อมการทำงานจริง การบูรณาการระบบหลายรายการเข้าด้วยกัน ทั้ง LINE API, Google Gemini AI, Jira API และ Google Sheets การประยุกต์ใช้ AI Large Language Model (LLM) สำหรับการวิเคราะห์ข้อความภาษาไทยและภาษาอังกฤษ การทำงานร่วมกับทีมงานจริง การรับ Feedback และการปรับปรุงระบบอย่างต่อเนื่อง (Iterative Development) การจัดการข้อมูลที่มีลักษณะ Unstructured Data ด้วยเทคนิค Regex และ AI Parsing การจัดการข้อมูลที่มีลักษณะ Unstructured Data ด้วยเทคนิค Regex และ AI Parsing

5.3 ข้อจำกัดของระบบ

ระบบในระยณีนี้อยู่มีข้อจำกัดบางประการที่ควรรับทราบ ดังนี้: ระบบยังไม่เชื่อมต่อกับ Jira ในสภาพแวดล้อม Production จริง ทดสอบเฉพาะใน UAT Environment เท่านั้น ความแม่นยำของ AI ขึ้นอยู่กับ System Prompt และรูปแบบข้อความของผู้ใช้งาน หากผู้ใช้งานพิมพ์ข้อความที่คลุมเครือมาก AI อาจจำแนกประเภทผิดพลาดได้ ระบบยังไม่รองรับการแจ้งปัญหาผ่านการส่งรูปภาพหรือไฟล์ รองรับเฉพาะข้อความตัวอักษรเท่านั้น Google Sheets ที่ใช้เป็น Cache มีข้อจำกัดด้านความเร็วเมื่อมีปริมาณข้อมูลสูง อาจต้องย้ายไปใช้ Database จริงในอนาคต

5.4 ข้อเสนอแนะสำหรับการพัฒนาในอนาคต

เพื่อให้ระบบมีประสิทธิภาพและครบสมบูรณ์ยิ่งขึ้น มีข้อเสนอแนะสำหรับการพัฒนาในระยะถัดไป ดังนี้:

เชื่อมต่อกับ Jira Production Environment: นำระบบออกสู่สภาพแวดล้อมจริงเพื่อให้ IT Service Desk สามารถรับ Ticket จากระบบได้โดยตรงเพิ่มความสามารถในการรับรูปภาพ: พัฒนา Multimodal AI Pipeline เพื่อให้ระบบวิเคราะห์รูปภาพ Screenshot ของข้อผิดพลาดที่ผู้ใช้งานส่งมาได้ Dashboard และ Report: พัฒนา Dashboard แสดงสถิติเคสแจ้งปัญหา เช่น จำนวนเคสต่อประเภท, เวลาเฉลี่ยในการแก้ไข และ Trend การแจ้งปัญหา เปลี่ยน Cache Storage: ย้ายจาก Google Sheets ไปใช้ Redis หรือ Database เพื่อรองรับ Volume สูงและลด Latency AI Feedback Loop: สร้างระบบ Feedback เพื่อให้เจ้าหน้าที่สามารถแก้ไขผลการจำแนกของ AI และนำข้อมูลไปปรับปรุง Prompt หรือ Fine-tune Model ต่อไป

5.5 บทสรุป

โครงการ LINE-2-JIRA เป็นตัวอย่างที่ชัดเจนของการนำเทคโนโลยี Workflow Automation และ AI มาประยุกต์ใช้เพื่อแก้ปัญหการจัดการข้อมูลในองค์กร ระบบที่พัฒนาขึ้นช่วยลดภาระงาน Manual ของเจ้าหน้าที่ ลดความผิดพลาดจากการบันทึกข้อมูล และเพิ่มความรวดเร็วในการตอบสนองต่อปัญหาของผู้ใช้งาน

การนำ n8n ร่วมกับ Google Gemini AI และการออกแบบ Workflow ที่รอบคอบ ทำให้ระบบสามารถรองรับความซับซ้อนของข้อมูลในสภาพแวดล้อมการทำงานจริงได้อย่างมีประสิทธิภาพ และเป็นต้นแบบสำคัญสำหรับการพัฒนาระบบ IT Automation ในองค์กรต่อไป

บรรณานุกรม

Atlassian. (2568). *Jira service management documentation*.

สืบค้นจาก <https://support.atlassian.com/jira-service-management-cloud/>

Google DeepMind. (2568). *Gemini API documentation*.

สืบค้นจาก <https://ai.google.dev/gemini-api/docs>

Groq, Inc. (2568). *Groq cloud API documentation*.

สืบค้นจาก <https://console.groq.com/docs>

LINE Corporation. (2568). *LINE messaging API documentation*.

สืบค้นจาก <https://developers.line.biz/en/docs/messaging-api/>

Mozilla Developer Network. (2568). *Regular expressions guide*.

สืบค้นจาก https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Regular_expressions

n8n GmbH. (2568). *n8n documentation*.

สืบค้นจาก <https://docs.n8n.io/>